COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 03: Convolutional Neural Networks

CE/Lecturer: Dr Trung Le (Clayton)

Lecturer: Dr Binh Nguyen (Clayton)

Lecturer: Dr Ting Fung Fung (Malaysia)

Lecturer: Dr Arghya Pal (Malaysia)

Lecturer: Dr Sailaja Rajanala (Malaysia)

Email: trunglm@monash.edu



GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI

Faculty of Information Technology, Monash University, Australia

Outline

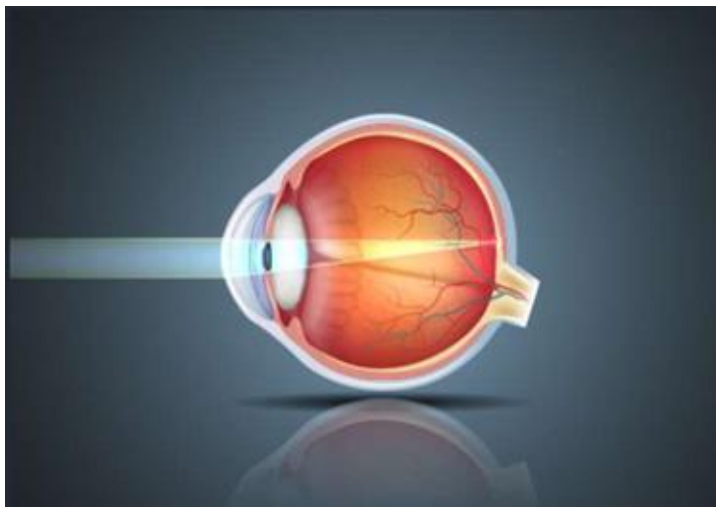
- Origin of human vision
- Fundamental of computer vision
- Convolutional Neural Networks
  - Convolutional operators in 1D, 2D, 3D
  - Standard CNN architecture:
    - Convolutional layer, Pooling layer, Fully-connected layer
  - CNN-based image recognition system
    - Batch norm and dropout layers
    - LeNet, AlexNet, and VGG

- Further reading recommendation
  - [HandsOn, chapter 14], [FDL, chapter 5], [Dive into DL, chapter 6].
  - "CS231n: Convolutional Neural Networks for Visual Recognition", Stanford University, 2016.

Origin of Human Vision

Human vision

- The vision begins with eyes, but truly take places in the brain.
- The collaboration between the eyes and the brain, called the **primary visual pathway**, is the reason we can make sense of the world around us.



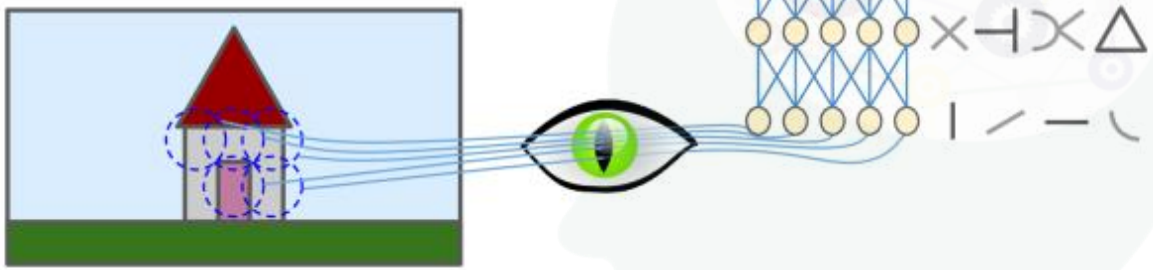

JUST BECAUSE YOU'RE MAD AT SOMEONE
DOESN'T MEAN YOU STOP LOVING THEM

Recognition is easy and fundamental

(Source: https://www.youtube.com/watch?v=40riCqvRoMs)



A boy
A boy is playing
A boy is playing in the park



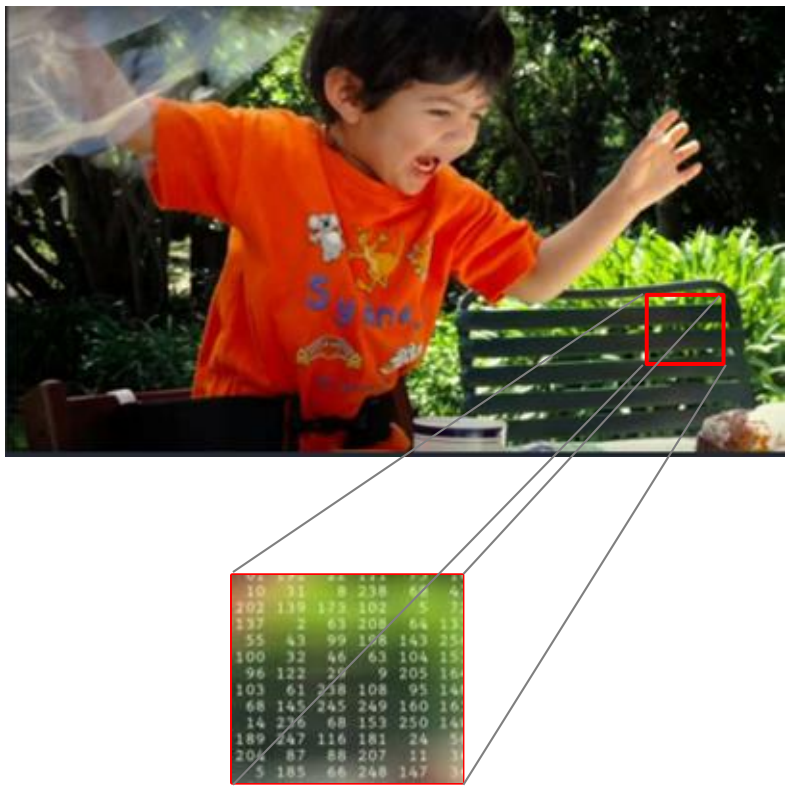
(Source: Hands On, Ch. 14)

Biological neurons *in the* ***visual cortex*** *respond to specific patterns in* ***small regions*** *of the* ***visual field*** *called* ***receptive fields****; as the visual signal makes its way through* ***consecutive brain modules****, neurons respond to more* ***complex patterns*** *in larger receptive fields.*

(Source: www.pngwing.com)

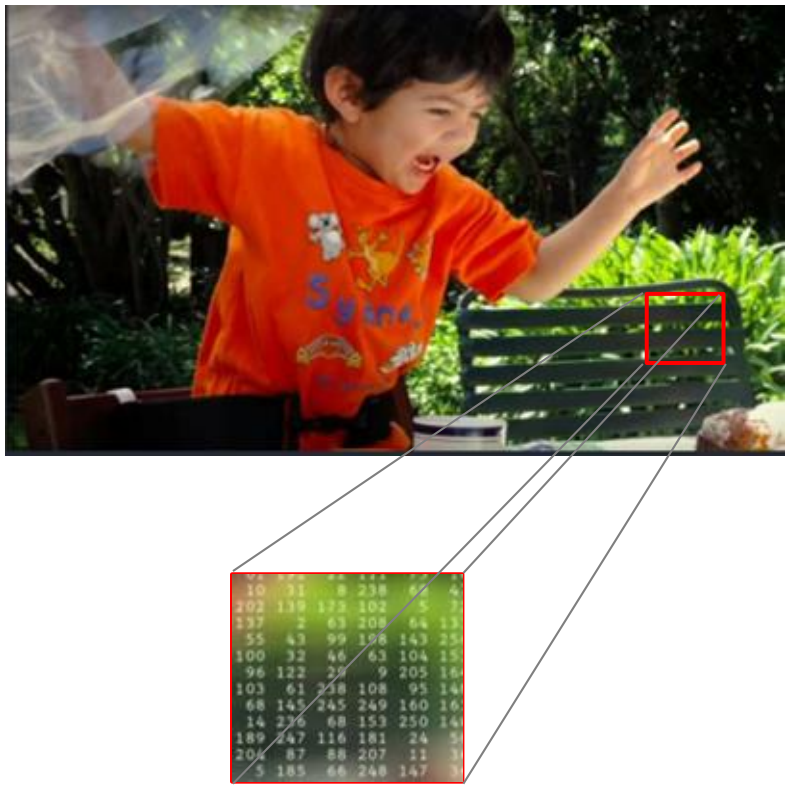
Recognition is much harder for machines

Human vision



Human vision vs computer vision

Human vision



pixel

Computer vision

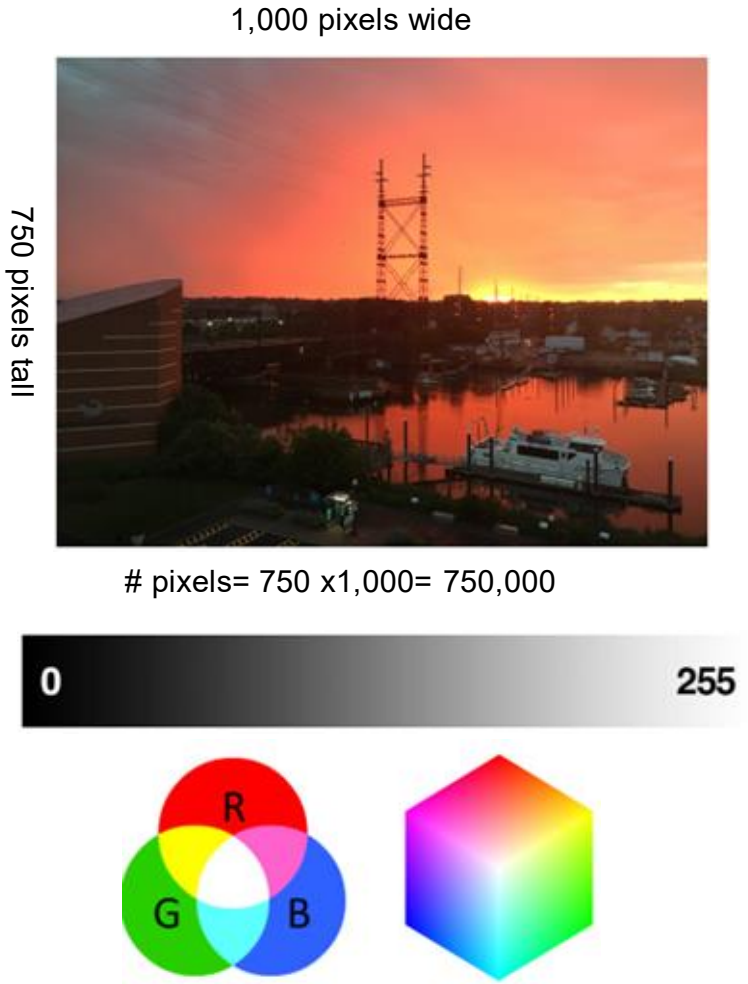


- Computer ‘see’ in a different way than we do. They are pixels!
- Every image can be represented as a 2-dimensional arrays of numbers, each represents a pixel.

Pixels: the building block of images

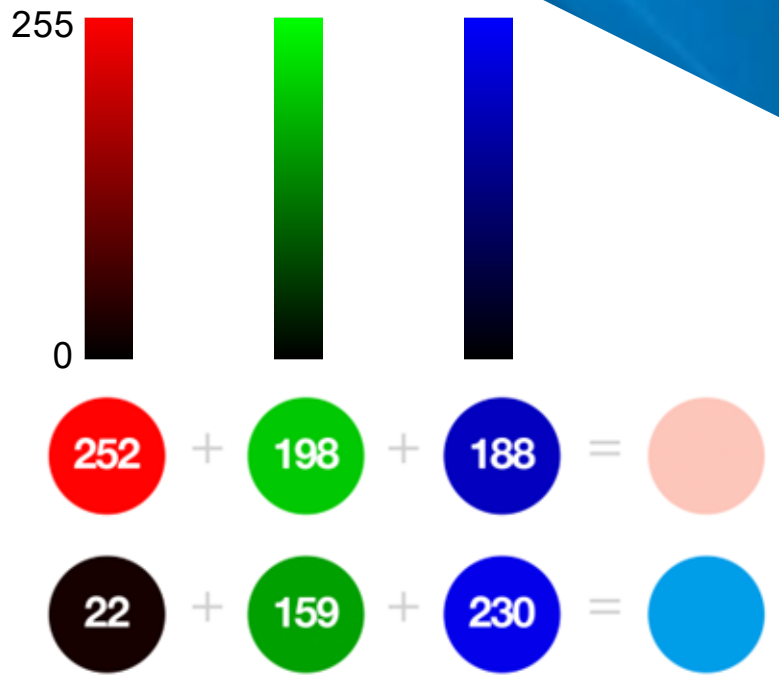
- Pixels are the raw building blocks of an image. Every image consists of a **set of pixels**.

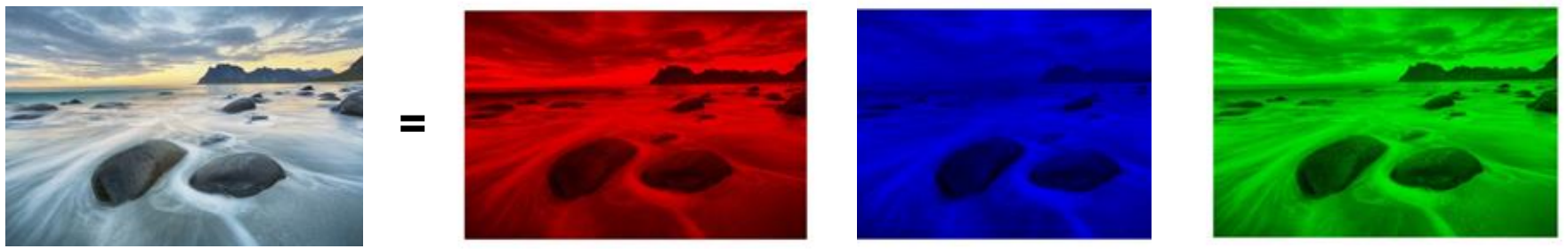
- Most pixels are represented in two ways:
  - Grayscale/single channel
    - A scalar between 0 (black) and 255 (white)
  - Color
    - Color pixels are represented in RGB system
      - Red, green, and blue channels



RGB colour space

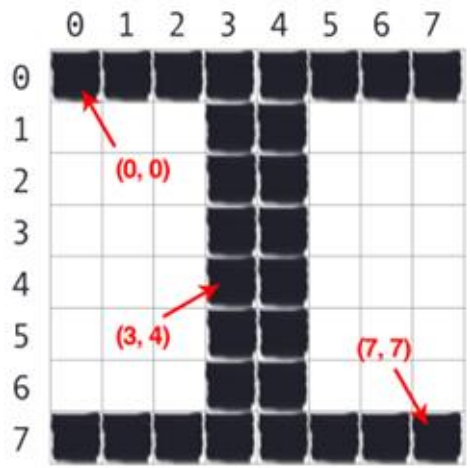
- A colour pixel is represented as a 3D tuple (R,G,B)
  - $R, G, B \in \{0,1, \ldots, 255\}$
- The number of possible colours
  - 256 x 256 x 256=16,777,216



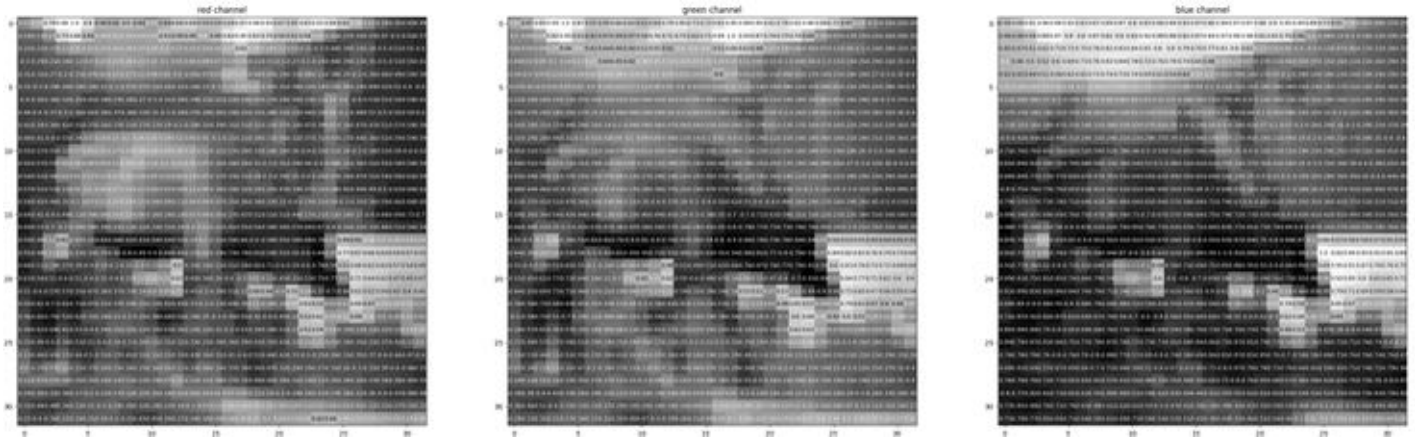


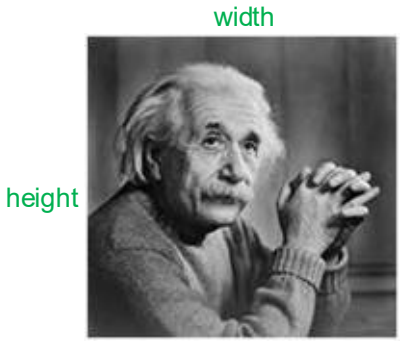
HSV and YCbCr are other ways to represent colors.

Image coordinate system

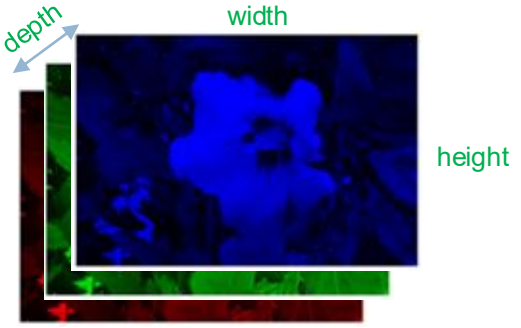


An image of letter I. Each pixel has a coordinate.





A grayscale image can viewed as a 2D tensor



A colour image can be viewed as a 3D tensor

Images as Tensor

```
# Load sample images
china = load_sample_image("china.jpg")[80:360, 70:390]
flower = load_sample_image("flower.jpg")[80:360, 130:450]
batch = np.array([china, flower], dtype=np.float32)
batch = np.transpose(batch, (0,3,1,2))
print(batch.shape)
```

```
(2, 3, 280, 320)
```

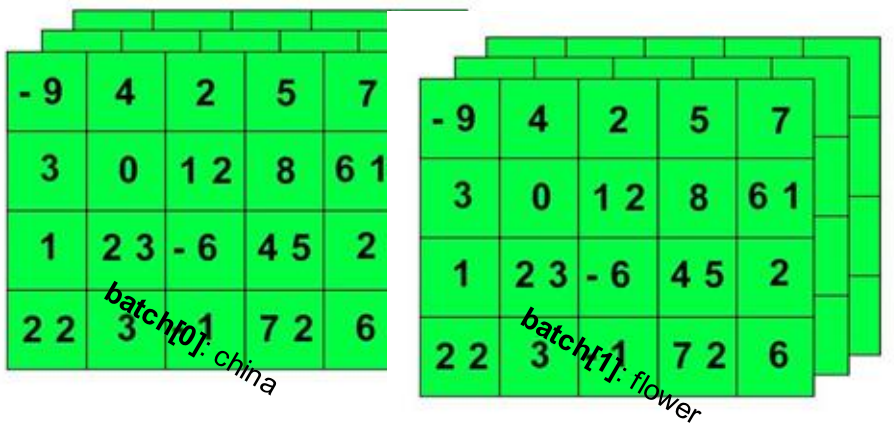
2 images

RGB channels

height

width



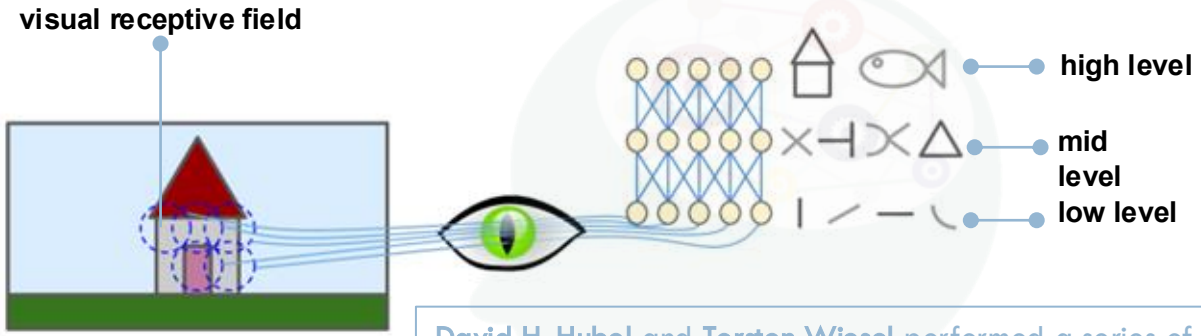




Convolutional Neural Networks



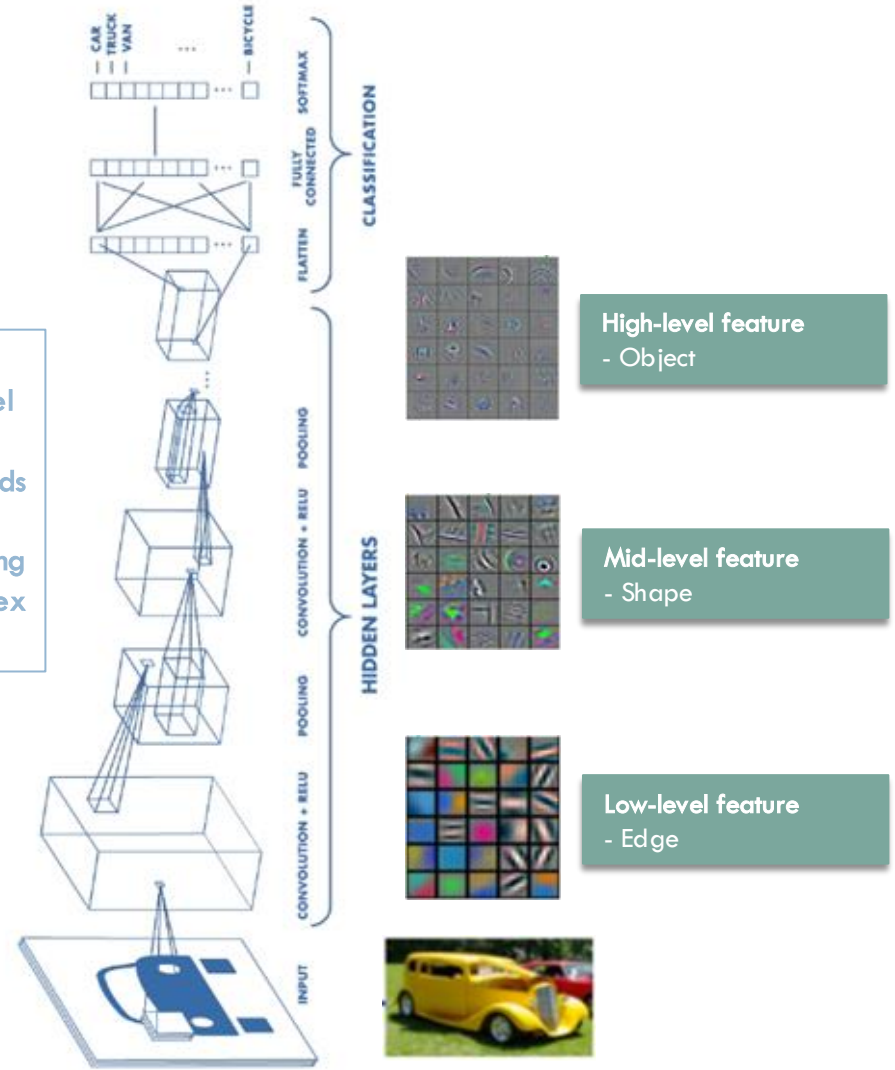
How to simulate human's vision?



(Source: Hands On, Ch. 14)

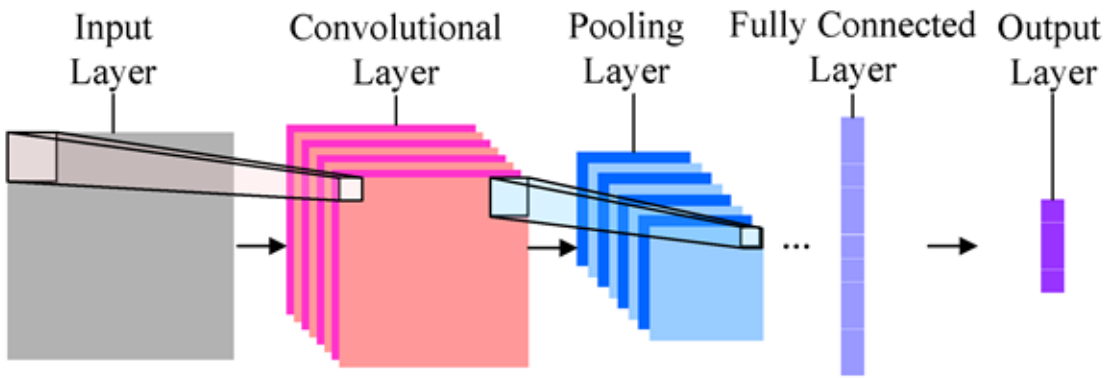
David H. Hubel and Torsten Wiesel performed a series of experiments on cats and monkeys in 1958 and 1959 (the Nobel Prize in Physiology or Medicine in 1981).

- Low-level neurons can capture small visual receptive fields with low-level patterns.
- Higher-level neurons are based on the outputs of neighboring lower-level neurons that can detect all sorts of complex patterns in any area of the visual field.



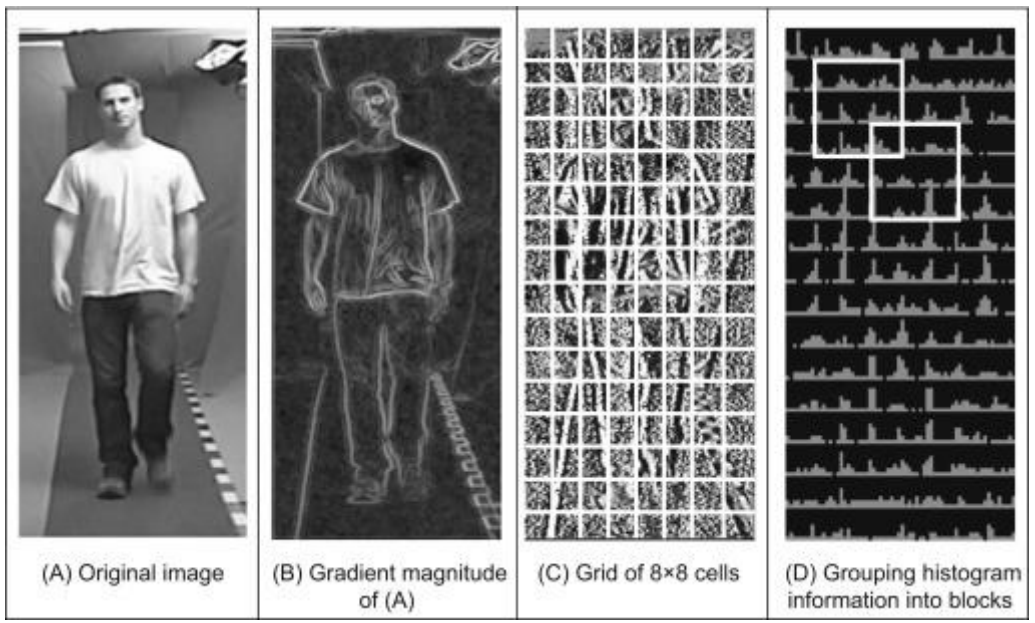
- Building-blocks of **Convolutional Neural Networks**
  - Convolutional layers
  - Pooling layers
    - Max pooling or average pooling
  - Fully-connected layers
  - Batch normalization layers

General architecture of CNNs

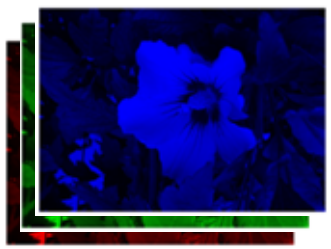


1. Convolutional layer
2. Pooling layer
3. Fully connected layer

Histogram of oriented gradients (HoG)



Why do we need Convolution layers?



[3,32,32]

What layers should we place in the middle?
Can we still use the linear layers?



softmax

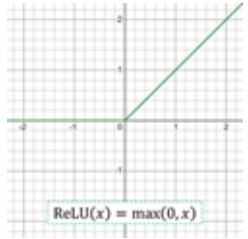


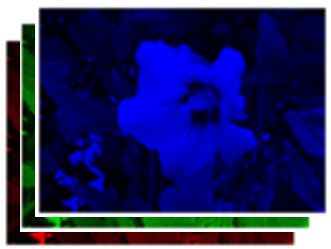
Output layer

10 neurons
for 10 classes

Why do we need Convolution layers?

ReLU (Rectified Linear Unit) is an activation function commonly used in neural networks.

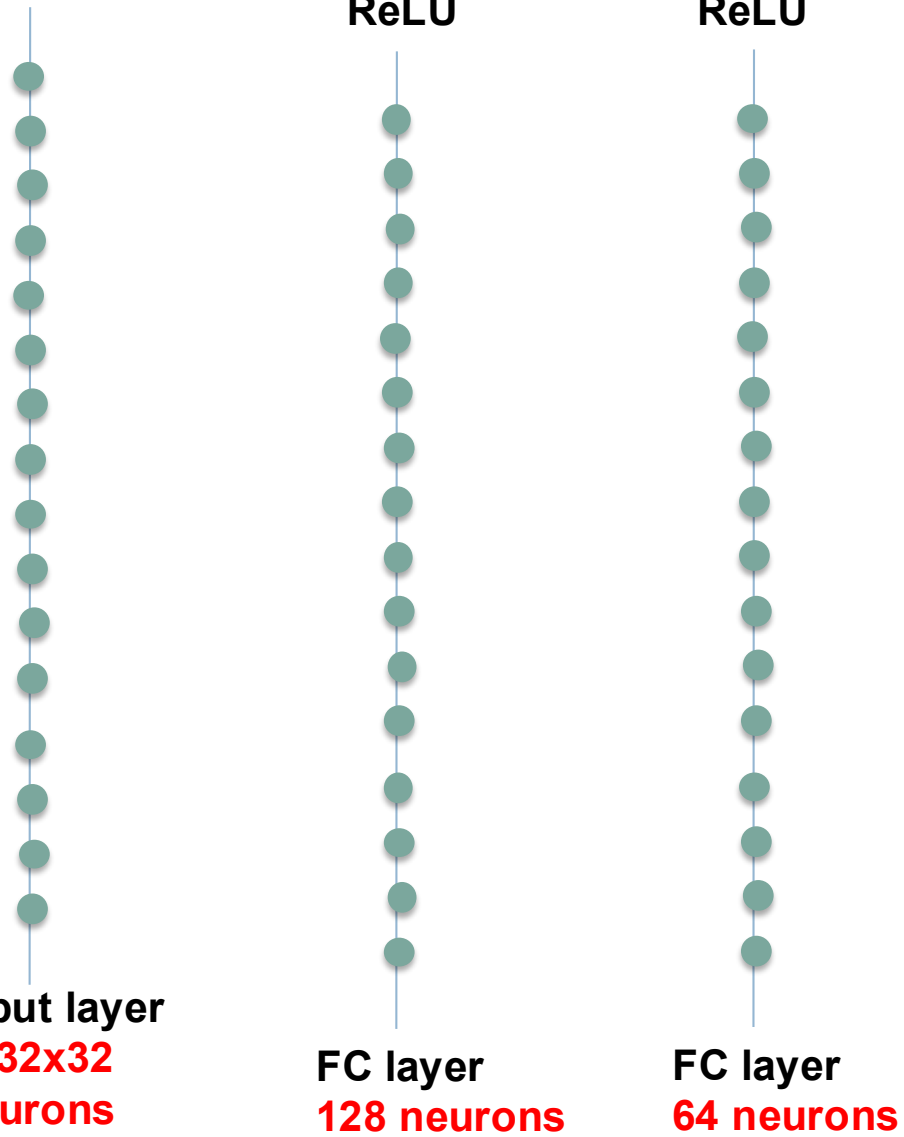




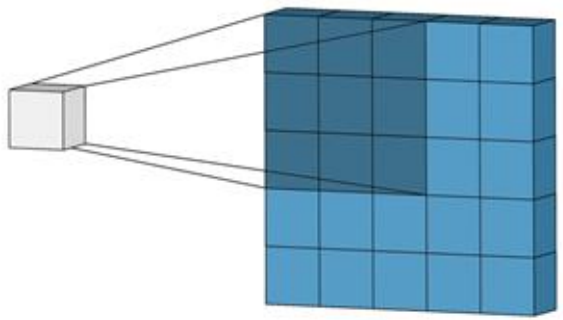
[3,32,32]

Flatten to [1, 3x32x32]

Cannot **take advantage of spatial Information** carried in pixels.



Convolutional Layer



Compression Represents Intelligence Linearly

Yuzhen Huang[*1] **Jinghan Zhang**[*1] **Zifei Shan**[2] **Junxian He**[1]
[1]The Hong Kong University of Science and Technology [2]Tencent
{yhuanghj,jzhangjv,junxianh}@cse.ust.hk

Abstract

There is a belief that learning to compress well will lead to intelligence (Hutter, 2006). Recently, language modeling has been shown to be equivalent to compression, which offers a compelling rationale for the success of large language models (LLMs): the development of more advanced language models is essentially enhancing compression which facilitates intelligence. Despite such appealing discussions, little empirical evidence is present for the interplay between compression and intelligence. In this work, we examine their relationship in the context of LLMs, treating LLMs as data compressors. Given the abstract concept of "intelligence", we adopt the average downstream benchmark scores as a surrogate, specifically targeting intelligence related to knowledge and commonsense, coding, and mathematical reasoning. Across 12 benchmarks, our study brings together 31 public LLMs that originate from diverse organizations. Remarkably, we find that LLMs' intelligence – reflected by average benchmark scores – almost *linearly* correlates with their ability to compress external text corpora. These results provide concrete evidence supporting the belief that superior compression indicates greater intelligence. Furthermore, our findings suggest that compression efficiency, as an unsupervised metric derived from raw text corpora, serves as a reliable evaluation measure that is linearly associated with the model capabilities. We open-source our compression datasets as well as our data collection pipelines to facilitate future researchers to assess compression properly.[3]

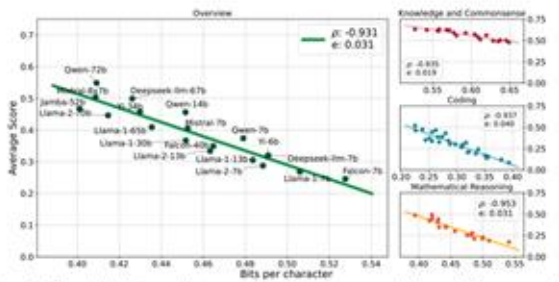


Figure 1: Correlation between the average benchmark scores and the models' compression efficiency evaluated with bits per character (BPC). BPC is computed on external raw corpora as listed in Table 1. Each point represents an LLM. **Left** shows the average benchmark score v.s. average BPC across, while **Right** focuses on three specific areas. ρ and ϵ denote the Pearson correlation coefficient and the root mean square error of the linear fit respectively.

Convolution operation

❖ W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

- **1D convolutional operation** $(W, x \in \mathbb{R}^{m \times 1})$
  - $W * x = \sum_{i=1}^{m} W_i x_i$

$$W = \begin{bmatrix} W_1 \\ \dots \\ W_m \end{bmatrix}, x = \begin{bmatrix} x_1 \\ \dots \\ x_m \end{bmatrix}$$

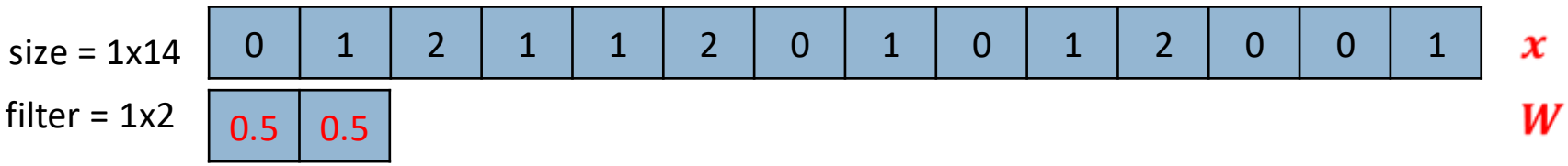
- **2D convolutional operation** $(W, x \in \mathbb{R}^{m \times n})$
  - $W * x = \sum_{i=1}^{m} \sum_{j=1}^{n} W_{ij} x_{ij}$

$$W = \begin{bmatrix} W_{11} & \dots & W_{1n} \\ \dots & \dots & \dots \\ W_{m1} & \dots & W_{mn} \end{bmatrix} \quad x = \begin{bmatrix} x_{11} & \dots & x_{1n} \\ \dots & \dots & \dots \\ x_{m1} & \dots & x_{mn} \end{bmatrix}$$

- **3D convolutional operation** $(W, x \in \mathbb{R}^{m \times n \times p})$
  - $W * x = \sum_{i=1}^{m} \sum_{j=1}^{n} \sum_{k=1}^{p} W_{ijk} x_{ijk}$



Convolution operation – 1D filter

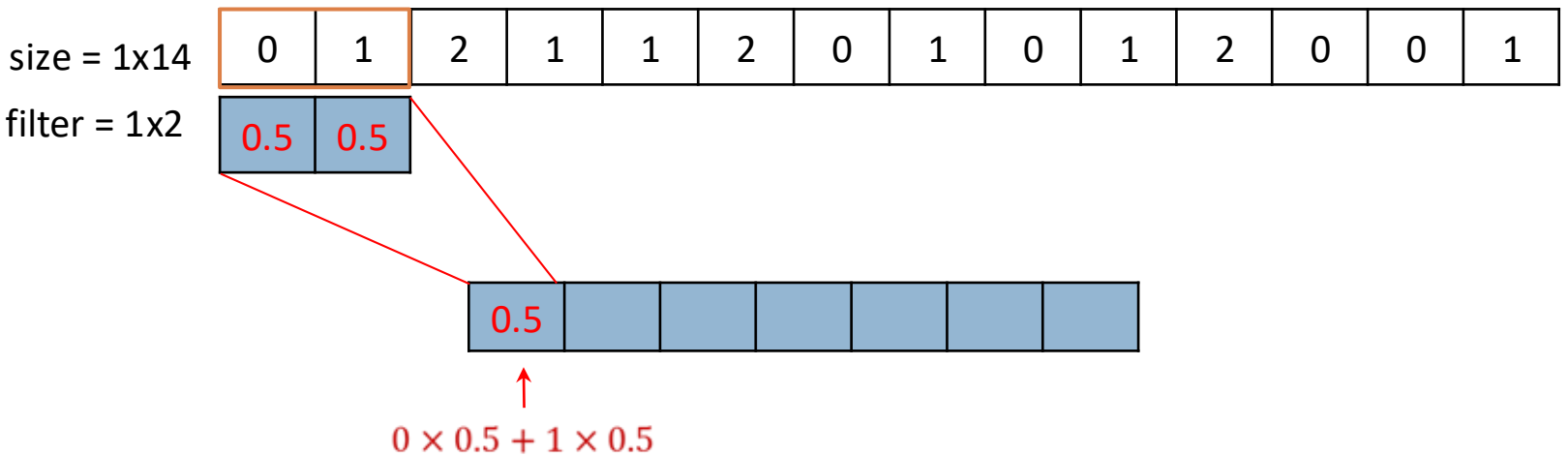


- W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

Convolution operation – 1D filter

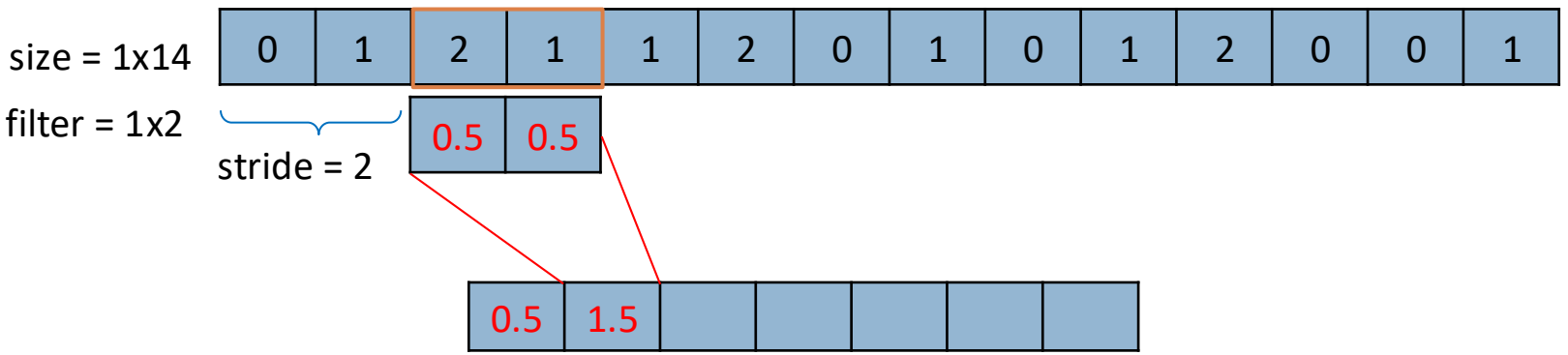


- W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

Convolution operation – 1D filter

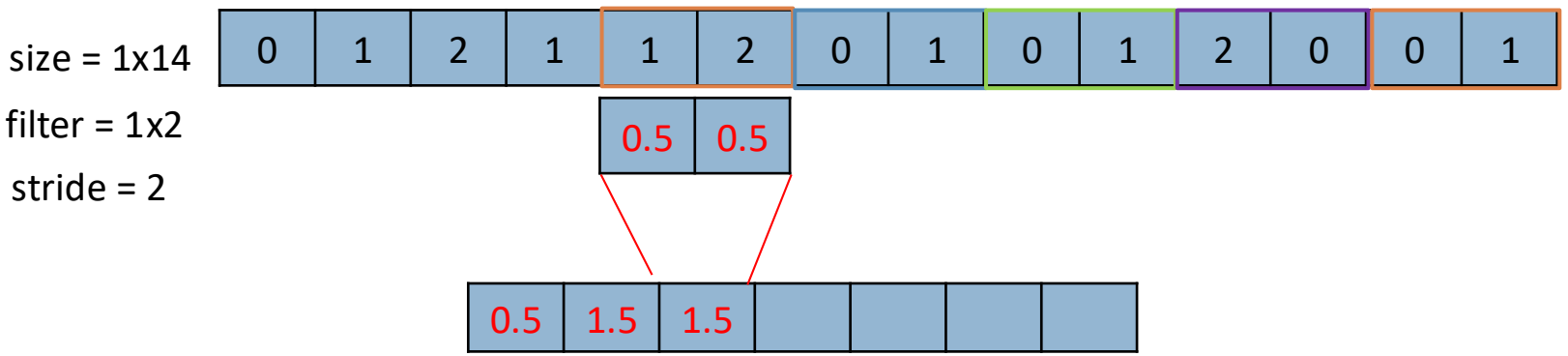


❖ W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

Convolution operation – 1D filter



❖ W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

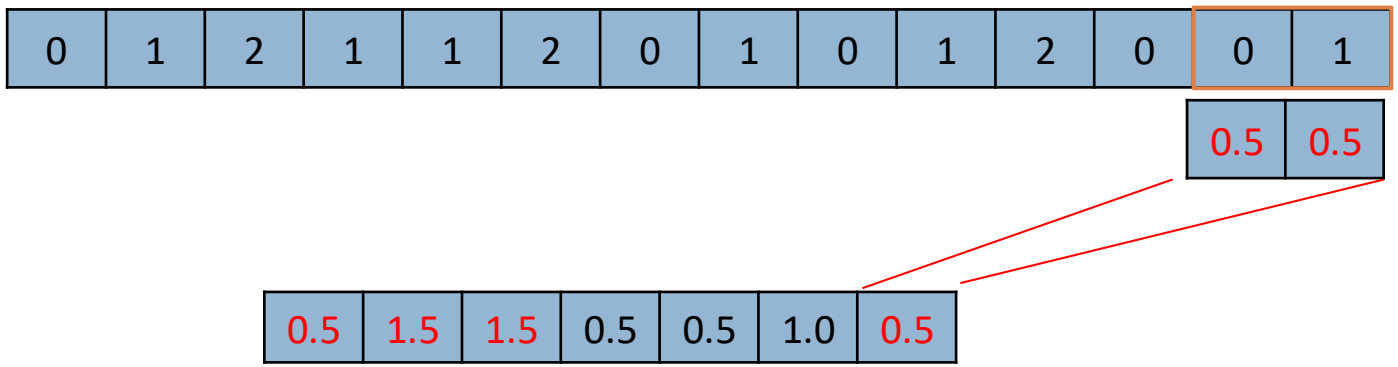
where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

Convolution operation – 1D filter

size = 14

filter = 2x1

stride = 2



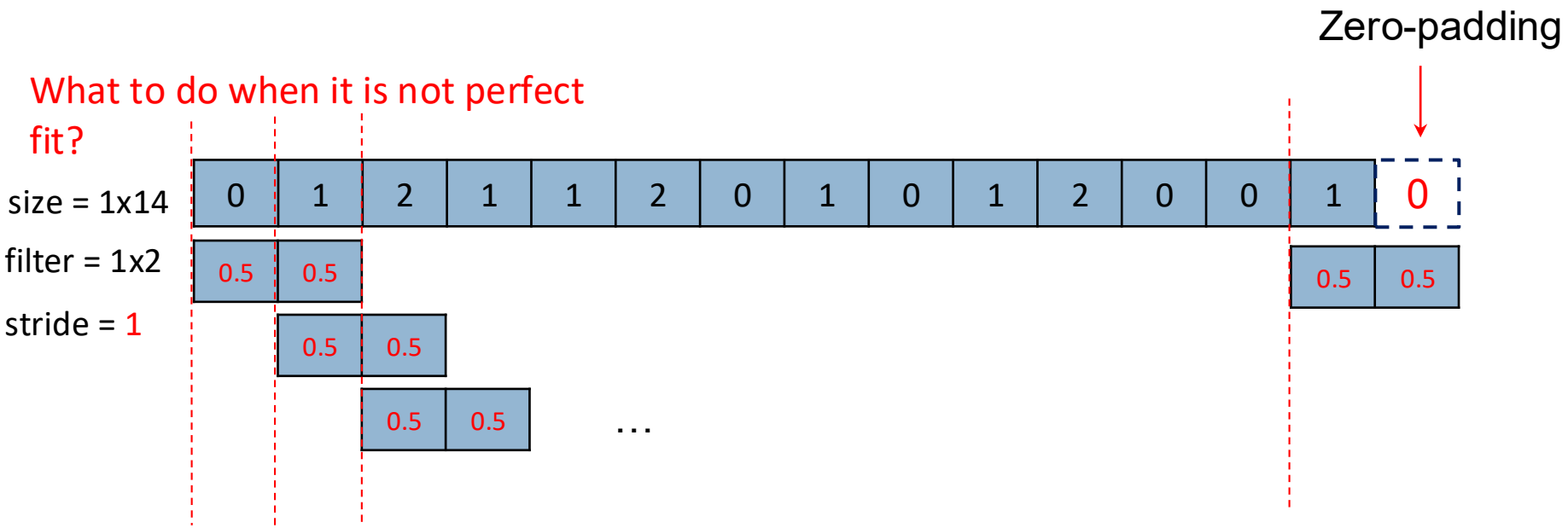
$$\text{new size} = 7 = \frac{14}{2} = \frac{\text{original size}}{\text{stride}}$$

- W and x are **two tensors** with the **same shape**, the **convolutional operation** between W and x defined as

$$W * x = sum(W \otimes x)$$

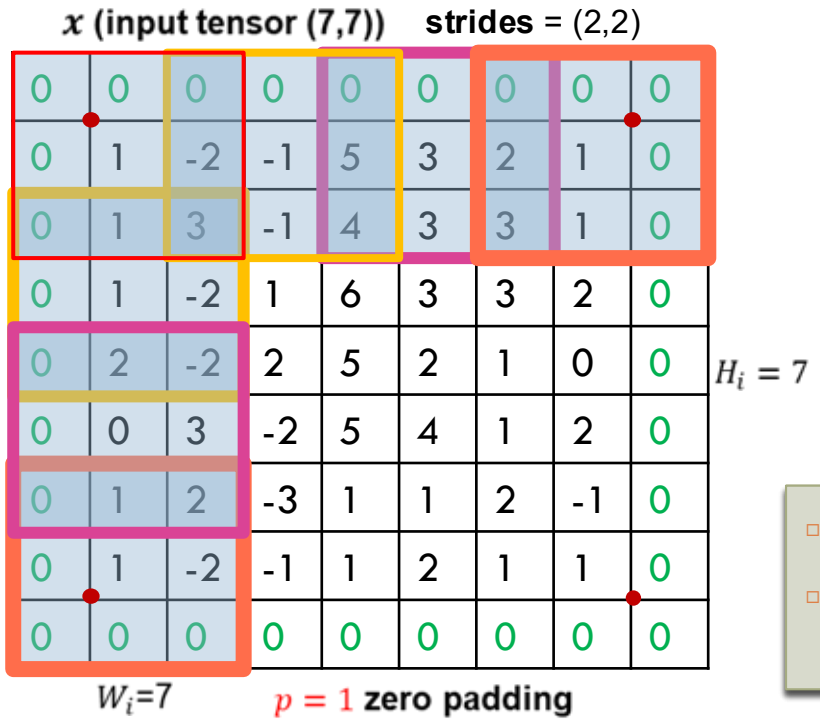
where $W \otimes x$ specifies the **element-wise product** and sum returns the **sum of all elements** in a tensor and multi-dimensional array.

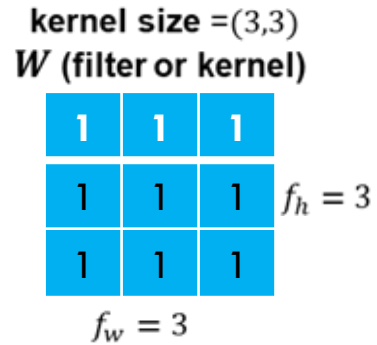
Convolution operation – 1D filter

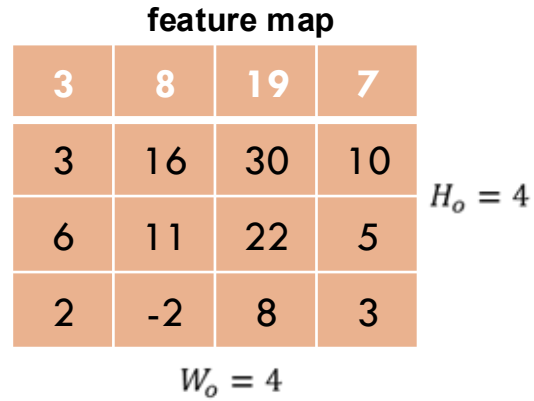


Exercise 1: what is the output size in this case?
Exercise 2: if stride = 3, do we need zero-padding, and if yes, what would be the output size?

Convolution layer with zero padding







- The **sliding window** moves from **left** to **right**, **top** to **bottom** with strides.
- We **convolve** the **filter** and the **sliding windows** to work out the **neurons** on the **feature map**.

Convolution layer with zero padding

x (input tensor (7,7)) **strides** = (2,2)

0	0	0	0	0	0	0	0	0
0	1	-2	-1	5	3	2	1	0
0	1	3	-1	4	3	3	1	0
0	1	-2	1	6	3	3	2	0
0	2	-2	2	5	2	1	0	0
0	0	3	-2	5	4	1	2	0
0	1	2	-3	1	1	2	-1	0
0	1	-2	-1	1	2	1	1	0
0	0	0	0	0	0	0	0	0

$H_i = 7$

W_i=7

$p = 1$ **zero padding**

kernel size =(3,3)

W **(filter or kernel)**

1	1	1
1	1	1
1	1	1

$f_h = 3$

$f_w = 3$

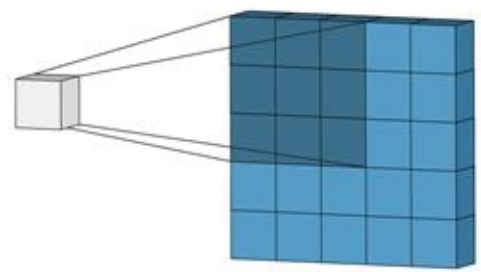
feature map

3	8	19	7
3	15	30	10
6	11	22	5
2	-2	9	3

$H_o = 4$

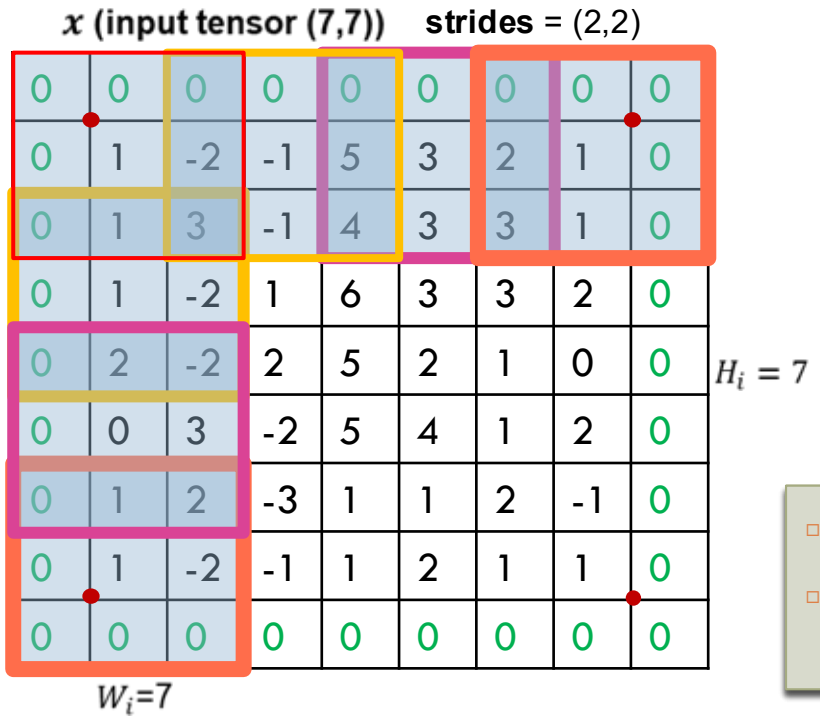
$W_o = 4$

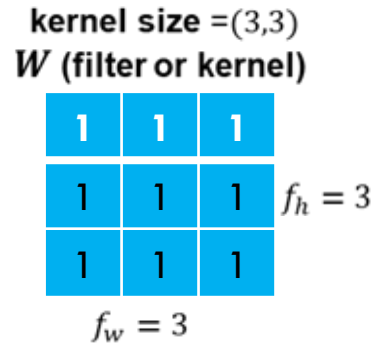
- The **sliding window** moves from **left** to **right**, **top** to **bottom** with strides.
- We **convolve** the **filter** and the **sliding windows** to work out the **neurons** on the **feature map**.

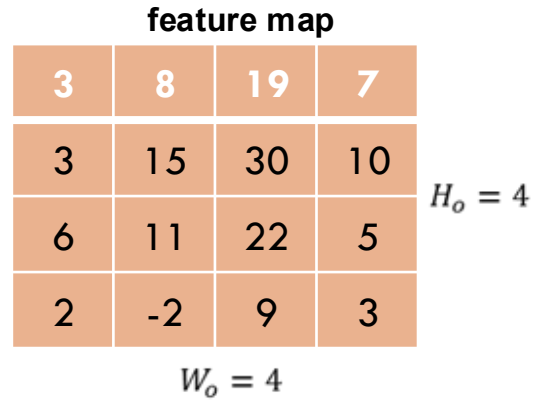


(Source: https://towardsdatascience.com/)

Convolution layer







- The **sliding window** moves from **left** to **right**, **top** to **bottom** with strides.
- We **convolve** the **filter** and the **sliding windows** to work out the **neurons** on the **feature map**.

- W_i, H_i: The width and height of the **input** image
- W_o, H_o: The width and height of the **output** image (feature map)

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- Our case: $W_o = \left\lfloor \frac{7+2\times1-3}{2} \right\rfloor + 1 = 4$ and $H_o = \left\lfloor \frac{7+2\times1-3}{2} \right\rfloor + 1 = 4$

Convolution layer

x (input tensor (7,8)) **strides** = (2,2)

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

$H_i = 7$

$W_i = 8$ **p=0 zero padding**

kernel size =(3,3)
***W* (filter or kernel)**

1	1	1
1	1	1
1	1	1

$f_h = 3$

$f_w = 3$

feature map

1	23	21
3	26	18
-1	8	13

$H_o = 3$

$W_o = 3$

Convolution layer

x (input tensor (7,8)) **strides** = (2,2)

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

$H_i = 7$

$W_i = 8$ **p=0 zero padding**

kernel size =(3,3)
***W* (filter or kernel)**

1	1	1
1	1	1
1	1	1

$f_h = 3$

$f_w = 3$

feature map

1	23	21
3	26	18
-1	8	13

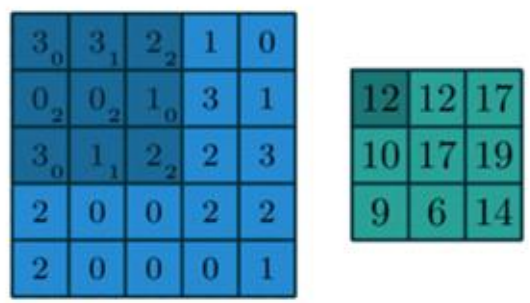
$H_o = 3$

$W_o = 3$

- W_i, H_i: The width and height of the **input** image
- W_o, H_o: The width and height of the **output** image (feature map)

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- Our case: $W_o = \left\lfloor \frac{8+0-3}{2} \right\rfloor + 1 = 3$ and $H_o = \left\lfloor \frac{7+0-3}{2} \right\rfloor + 1 = 3$



(Source: Internet)

Convolution 2D without zero padding

$H_i = 7$

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

x = tensor(7,8) $W_i = 8$

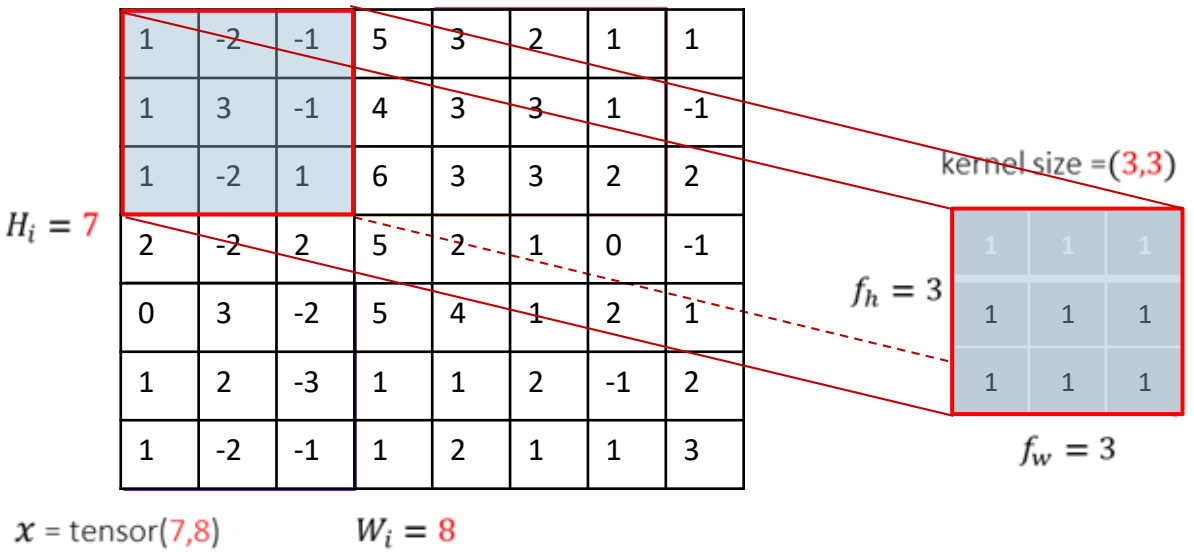
kernel size =(3,3)

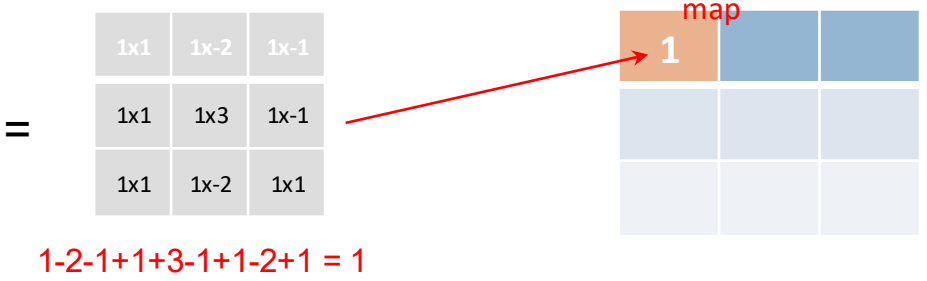
$f_h = 3$

1	1	1
1	1	1
1	1	1

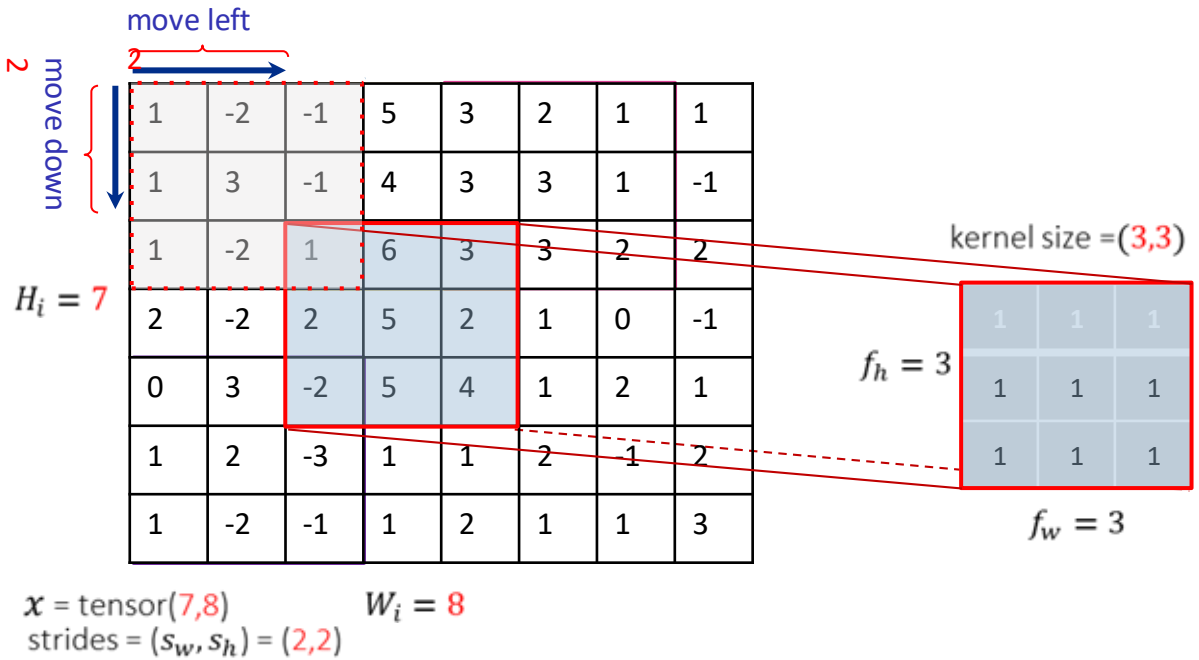
$f_w = 3$

Convolution 2D without zero padding





Convolution 2D without zero padding

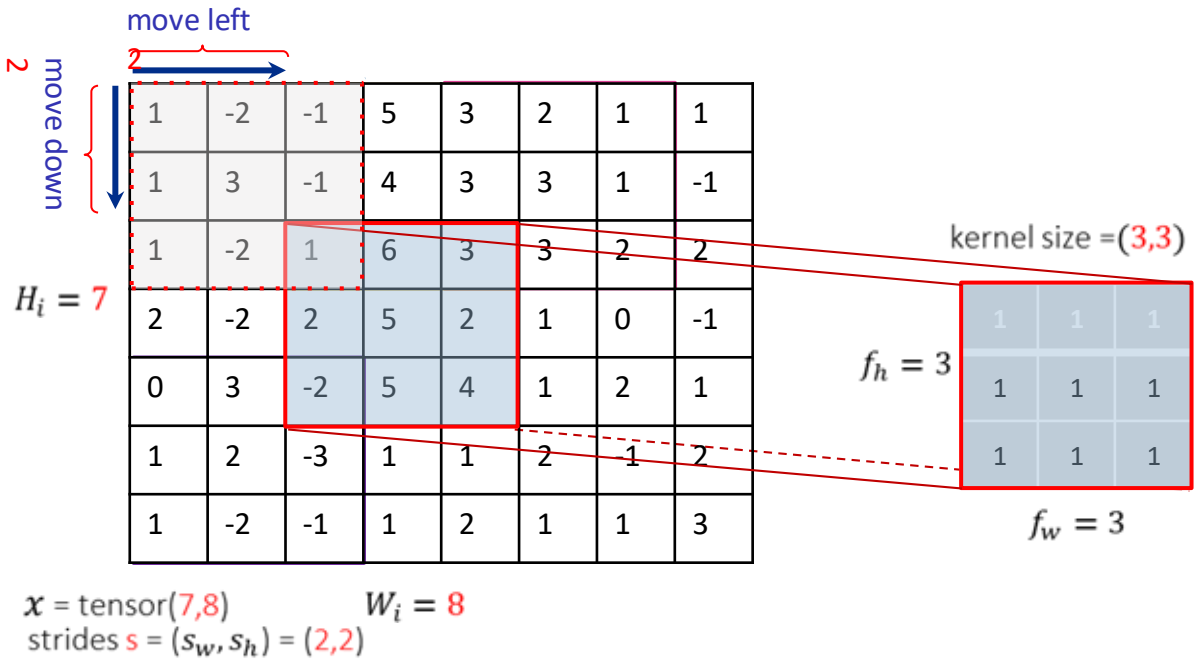


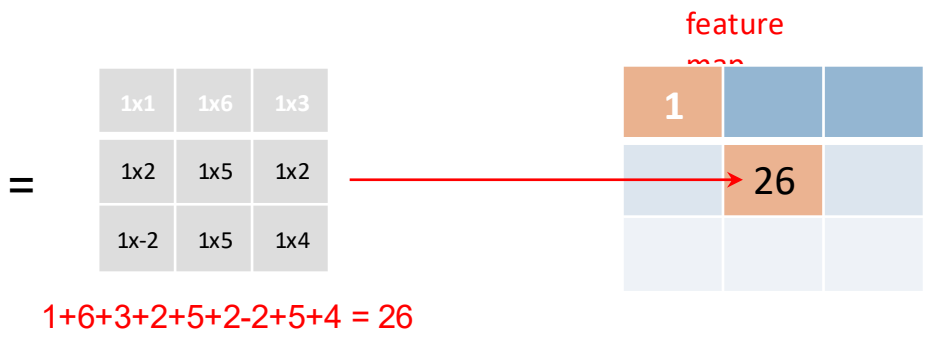
=



1+6+3+2+5+2-2+5+4 = 26

Convolution 2D without zero padding





Convolution 2D without zero padding

$H_i = 7$

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

$W_i = 8$

x = tensor(7,8)

strides s = (s_w, s_h) = (2,2)

kernel size =(3,3)

$f_h = 3$

1	1	1
1	1	1
1	1	1

$f_w = 3$

feature map

1		
3	26	

Convolution 2D without zero padding

$H_i = 7$

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

x = tensor(7,8) $W_i = 8$

strides s = (s_w, s_h) = (2,2)

kernel size =(3,3)

$f_h = 3$

1	1	1
1	1	1
1	1	1

$f_w = 3$

feature map

1		
3	26	
-1		

Convolution 2D without zero padding

$H_i = 7$

1	-2	-1	5	3	2	1	1
1	3	-1	4	3	3	1	-1
1	-2	1	6	3	3	2	2
2	-2	2	5	2	1	0	-1
0	3	-2	5	4	1	2	1
1	2	-3	1	1	2	-1	2
1	-2	-1	1	2	1	1	3

$W_i = 8$

x = tensor(7,8)

strides s = (s_w, s_h) = (2,2)

kernel size =(3,3)

$f_h = 3$

1	1	1
1	1	1
1	1	1

$f_w = 3$

feature map

$H_o = 3$

1	23	21
3	26	17
-1	8	13

$W_o = 3$

- W_i, H_i: The width and height of the **input** image
- W_o, H_o: The width and height of the **output** image (feature map)

$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- Our case: $W_o = \left\lfloor \frac{8+0-3}{2} \right\rfloor + 1 = 3$ and $H_o = \left\lfloor \frac{7+0-3}{2} \right\rfloor + 1 = 3$

Convolution 2D with zero padding

0	0	0	0	0	0	0	0	0
0	1	-2	-1	5	3	2	1	0
0	1	3	-1	4	3	3	1	0
0	1	-2	1	6	3	3	2	0
0	2	-2	2	5	2	1	0	0
0	0	3	-2	5	4	1	2	0
0	1	2	-3	1	1	2	-1	0
0	1	-2	-1	1	2	1	1	0
0	0	0	0	0	0	0	0	0

padding = 1

padding = 1

x = tensor(7,7)
strides s = (s_w, s_h) = (2,2)
zero-padding $p = 1$

kernel size =(3,3)

$f_h = 3$

1	1	1
1	1	1
1	1	1

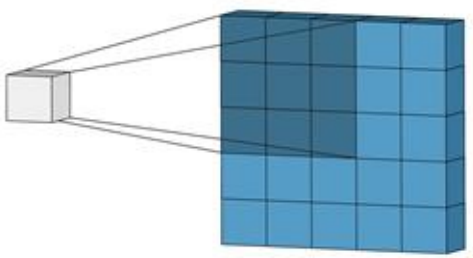
$f_w = 3$

feature

3	8	20	7
3	16	30	10
6	13	22	7
2	-2	7	3

$H_o = 4$

$W_o = 4$



- W_i, H_i: The width and height of the **input** image
- W_o, H_o: The width and height of the **output** image (feature map)

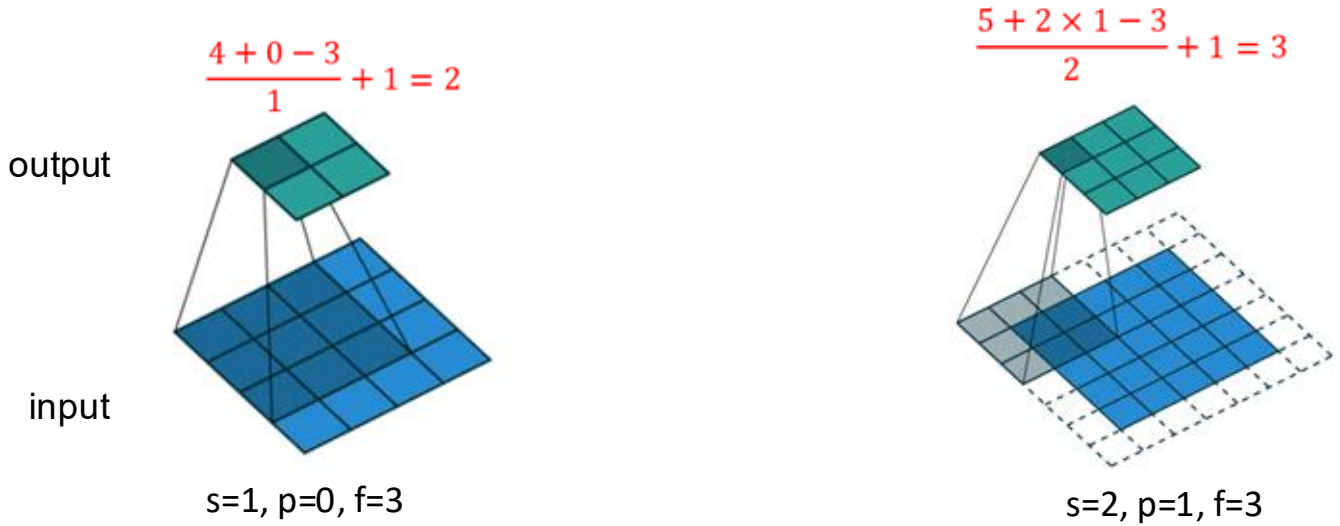
$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s_w} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i + 2p - f_h}{s_h} \right\rfloor + 1$$

- Our case: $W_o = \left\lfloor \frac{7+2\times1-3}{2} \right\rfloor + 1 = 4$ and $H_o = \left\lfloor \frac{7+2\times1-3}{2} \right\rfloor + 1 = 4$

(Source: https://towardsdatascience.com/)

Convolution layer

Example



s=1, p=0, f=3

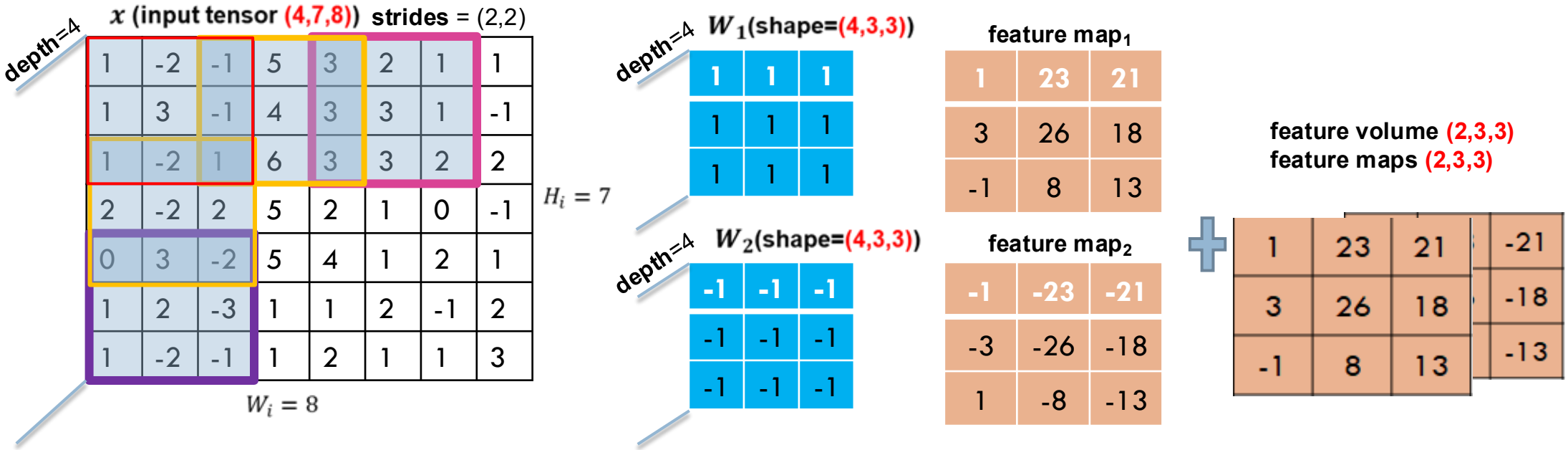
s=2, p=1, f=3

s = stride, p = padding, f = filter size (square) (Source: An intuitive guide to Convolutional Neural Networks)

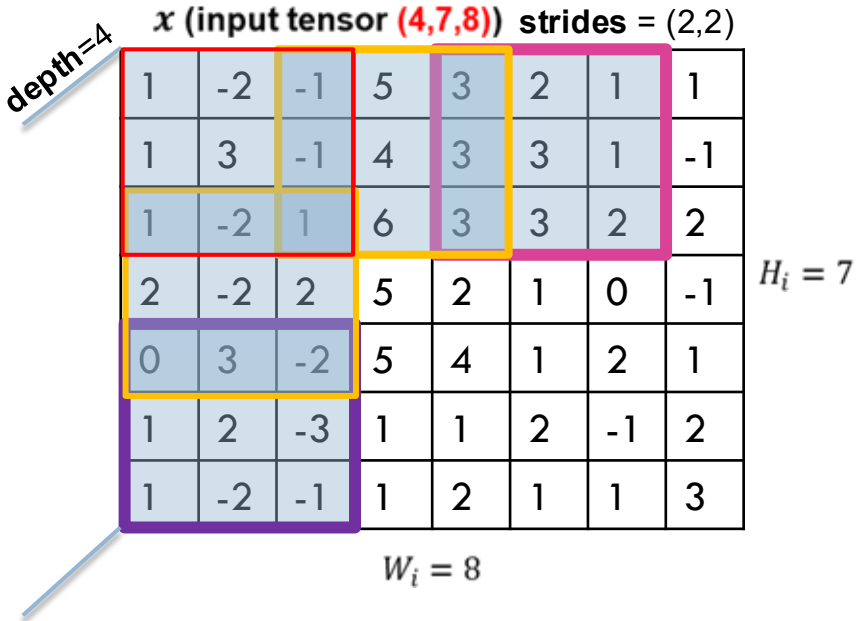
- W_i, H_i: The width and height of the **input image/tensor**
- W_o, H_o: The width and height of the **output image/tensor** or **feature map**

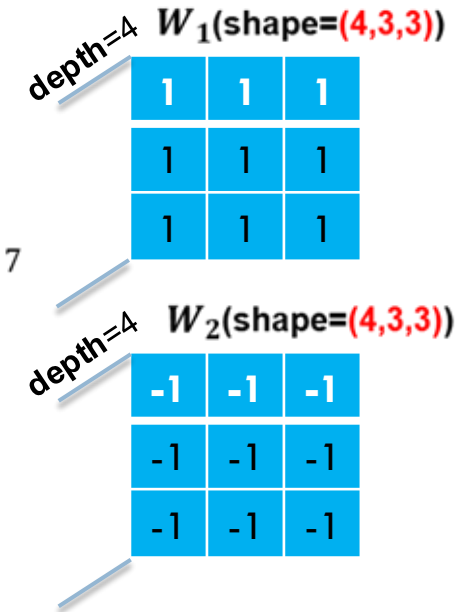
$$W_o = \left\lfloor \frac{W_i+2p-f_w}{s_w} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i+2p-f_h}{s_h} \right\rfloor + 1$$

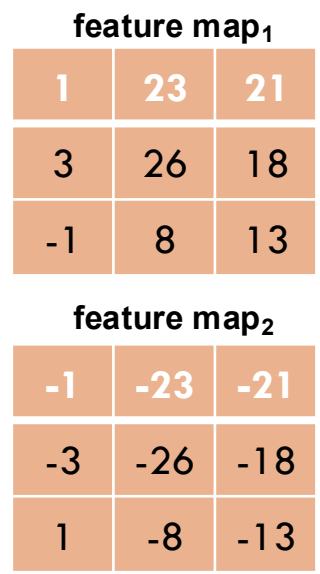
Convolution layer with multiple filters and feature maps

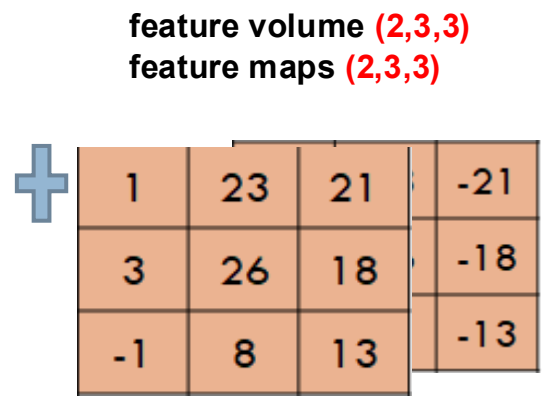


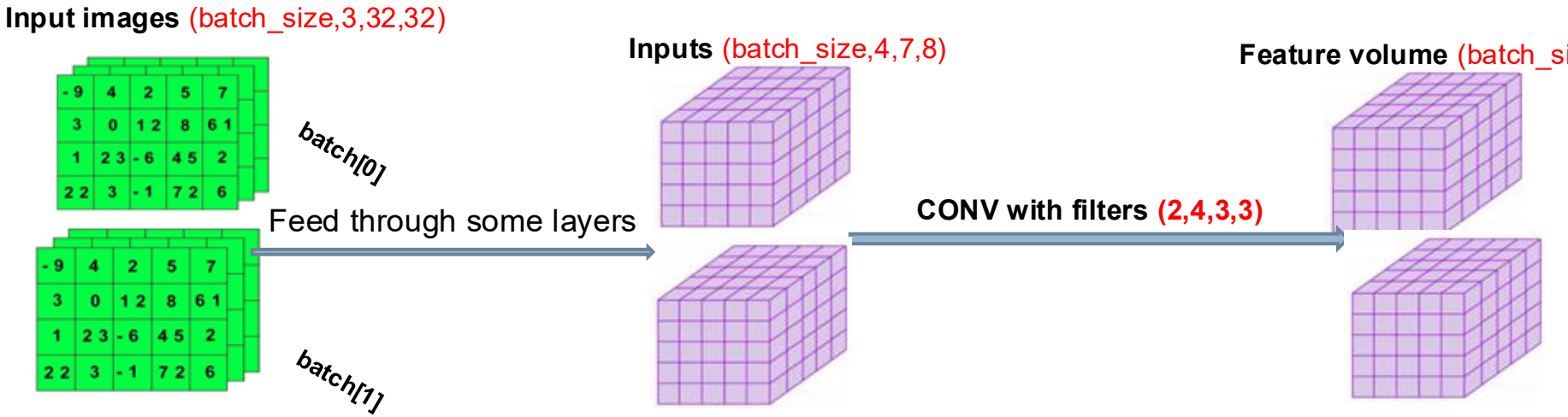
Convolution layer with multiple filters and feature maps












```
output = torch.nn.functional.conv2d(input = batch_tensor, weight= filters_tensor, stride =(2,2), padding= 3)
```

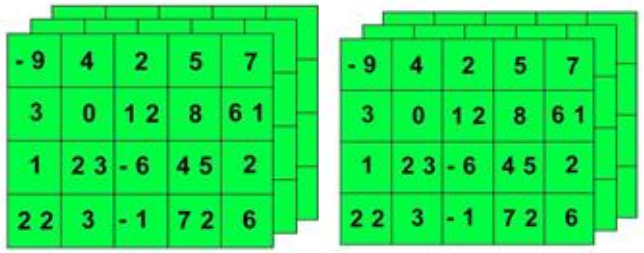
batch_size, in_channels , height, width,

out_channels, in_channels, fil_height, fil_width
,

Example of convolutional operation with PyTorch

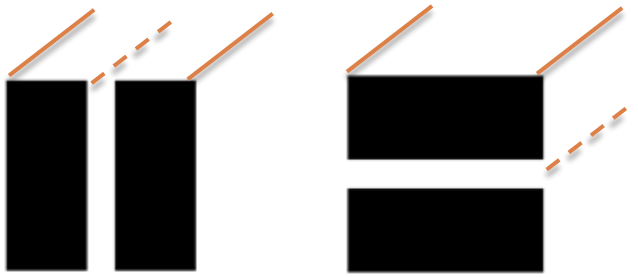
```
# Load sample images
china = load_sample_image("china.jpg")[80:360, 70:390]
flower = load_sample_image("flower.jpg")[80:360, 130:450]
batch = np.array([china, flower], dtype=np.float32)
batch = np.transpose(batch, (0,3,1,2))
print(batch.shape)
```

```
(2, 3, 280, 320)
```




```
# Create 2 filters
filters = np.zeros(shape=(2, 7, 7, channels), dtype=np.float32)
filters[0, 3, :, :] = 1  # vertical line, why?
filters[1, :, 3, :] = 1  # horizontal line, why?
filters = np.transpose(filters, (0,3,1,2))
filters.shape
```

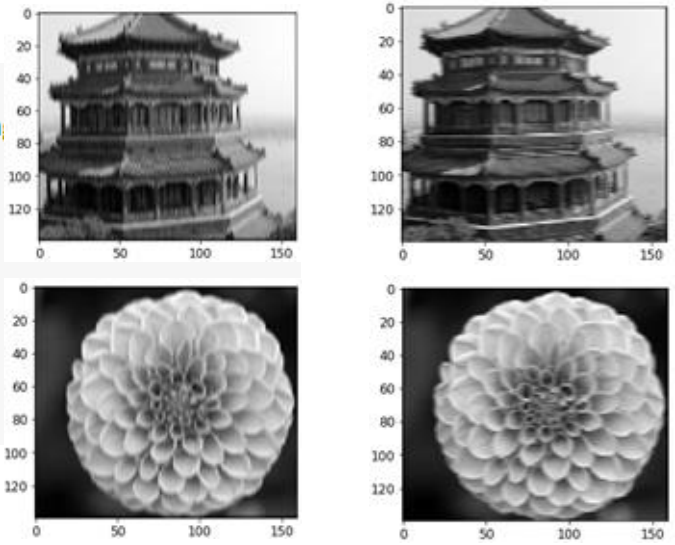
```
(2, 3, 7, 7)
```



```
output = torch.nn.functional.conv2d(input = batch_tensor, weight= filters_tensor,
                                    stride =(2,2), padding= 3) #padding means number of paddin
output = output.numpy()

print("Output shape:" + str(output.shape))

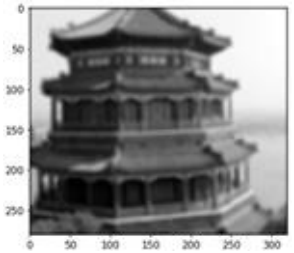
plot_image(output[0, 0, :, :], axis=True) # plot 1st image's 1nd feature map, channel 0
plot_image(output[0, 1, :, :], axis=True) # plot 1st image's 2nd feature map, channel 1
plot_image(output[1, 0, :, :], axis=True) # plot 2nd image's 1nd feature map, channel 0
plot_image(output[1, 1, :, :], axis=True) # plot 2nd image's 2nd feature map, channel 1
```



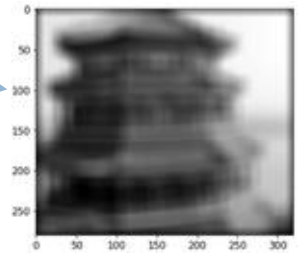
Effects of filters/kernels to images



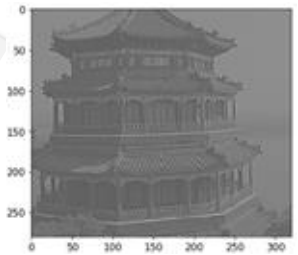
```
smallBlur = np.ones((7, 7), dtype="float") * (1.0 / (7 * 7))
```



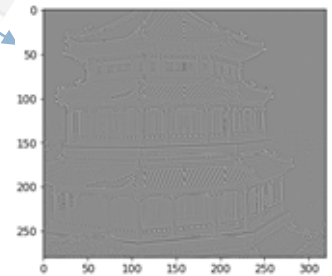
```
largeBlur = np.ones((21, 21), dtype="float") * (1.0 / (21 * 21))
```



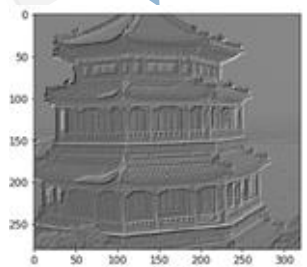
```
# construct a sharpening filter
sharpen = np.array(([0, -1, 0],
                    [-1, 5, -1],
                    [0, -1, 0]), dtype="int")
```



```
# construct the Laplacian kernel used to detect edge-like
# regions of an image
laplacian = np.array(([0, 1, 0],
                      [1, -4, 1],
                      [0, 1, 0]), dtype="int")
```



```
# construct an emboss kernel
emboss = np.array(([-2, -1, 0],
                   [-1, 1, 1],
                   [0, 1, 2]), dtype="int")
```

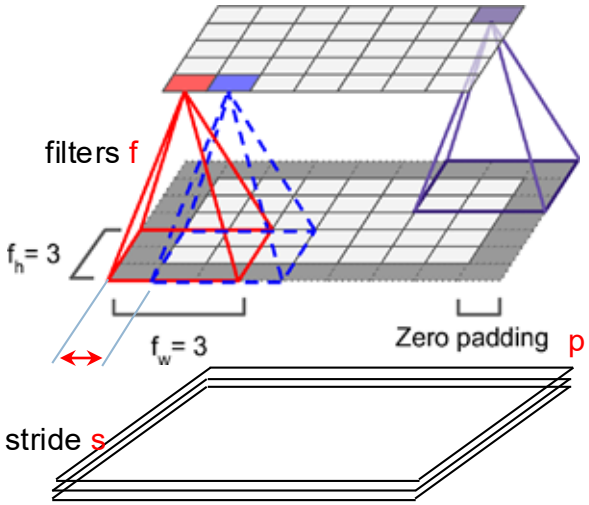


CNNs learn meaningful filters automatically from data!

Summary: full convolutional layer

- Input is a volume, e.g., an RGB image:
  - Width W_i
  - Height H_i
  - Depth D
  - Zero padding
- Output is another volume
  - Width W_o
  - Height H_o
  - Depth K (= number of filters)
- W_i, H_i: The width and height of the input image
- W_o, H_o: The width and height of the output image

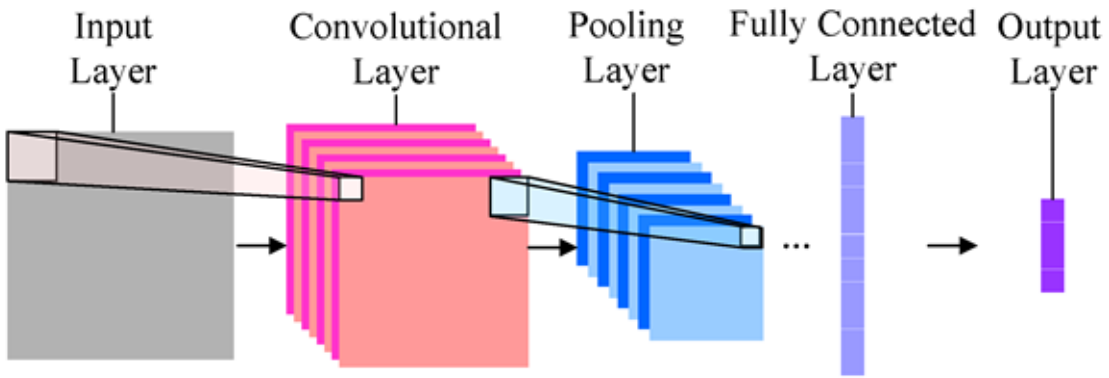
$$W_o = \left\lfloor \frac{W_i + 2p - f_w}{s} \right\rfloor + 1 \text{ and } H_o = \left\lfloor \frac{H_i + 2p - f_h}{s} \right\rfloor + 1$$





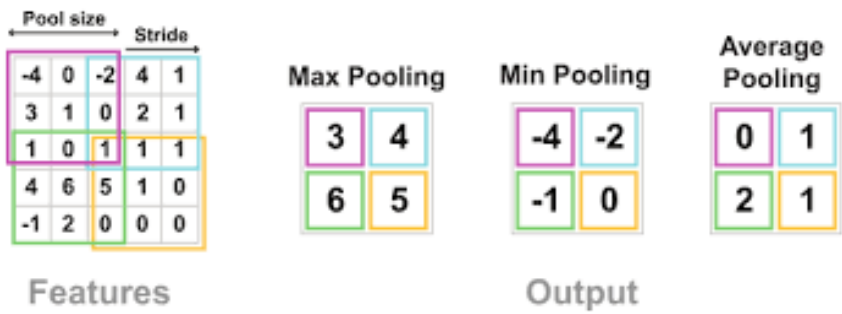
s = stride, p = padding, f = filter size (square)

General architecture of CNNs



1. Convolutional layer
2. Pooling layer
3. Fully connected layer

Pooling layer



Pooling operation

- Makes the representations smaller and more manageable
- Subsample the image
- Operates over each feature map independently

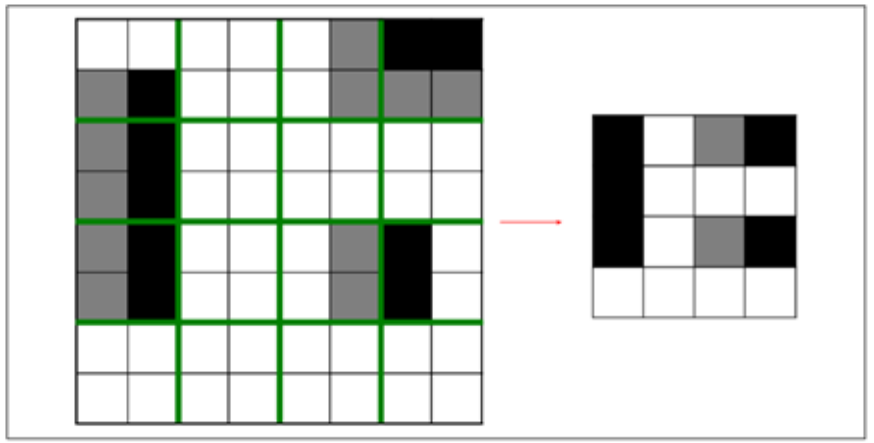


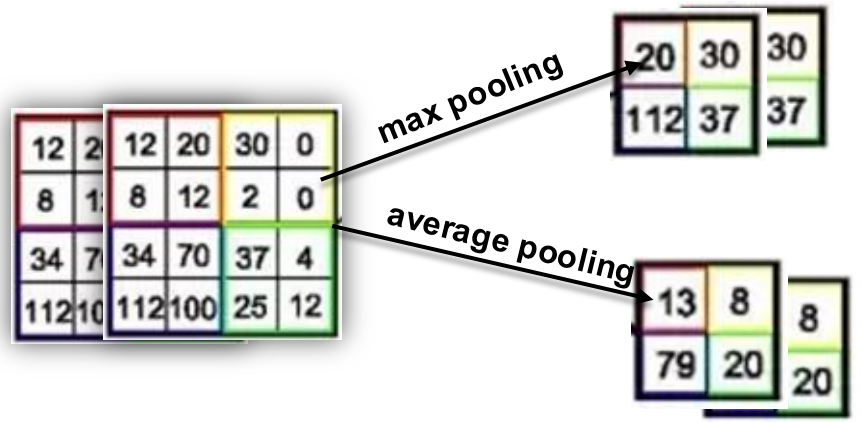
Figure 5-13. An illustration of how max pooling significantly reduces parameters as we move up the network

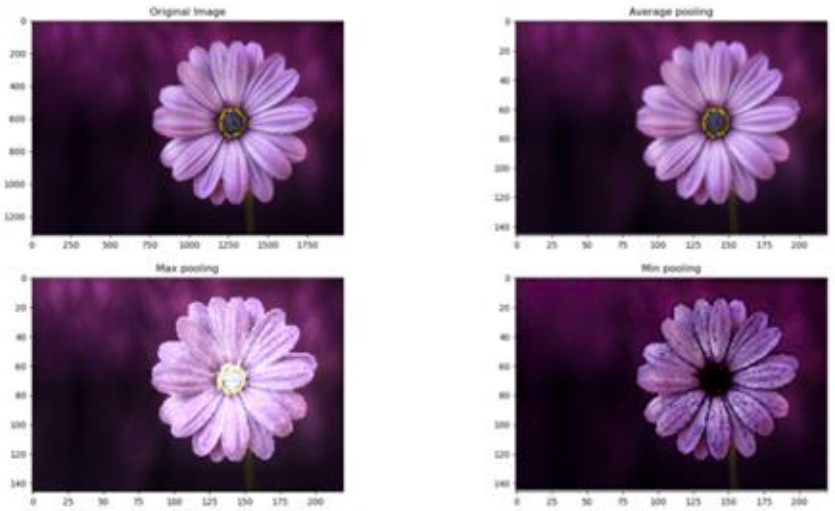
(Source: FDL Ch. 5)

Max/average pooling

- Max/average-pooling is **locally invariant** and applied **independently** to **each feature map/channel.**

- In the SOTA CNNs (ResNet, GoogleNet, VGG), we employ the pooling layer with
  - Kernel/filter **size = (2,2), strides = (2,2), padding= 0**
  - Therefore, the output tensor has the size
    - $W_o = \left\lfloor \frac{224+0-2}{2} \right\rfloor + 1 = 112$
    - $H_o = \left\lfloor \frac{224+0-2}{2} \right\rfloor + 1 = 112$
  - With this setting, we **down-sample** the **input size** by **2**

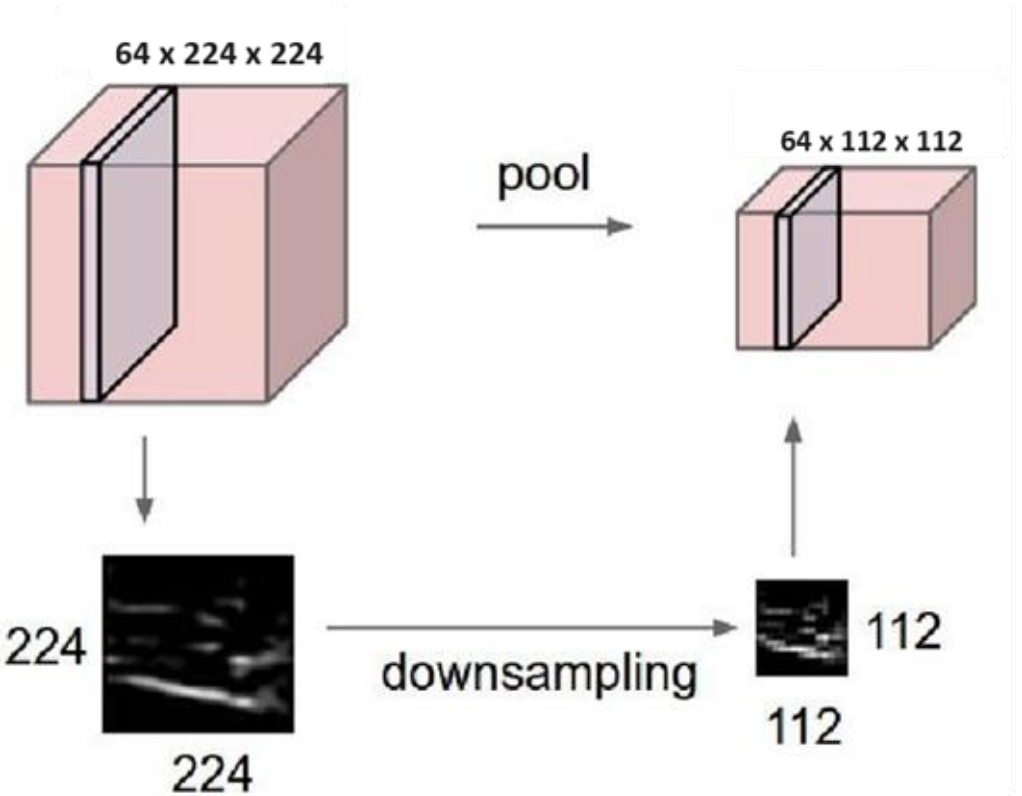




(Source: medium.com)

Pooling operation

- Makes the representations smaller and more manageable
- Operates over each channel independently



Example of pooling with PyTorch

Max pooling

```
output = torch.nn.functional.max_pool2d(input = batch_tensor,
                                        kernel_size=(2,2), stride= (2,2))
output = output.numpy()
print(batch[0].shape)
print(output[0].shape)
plot_image(batch[0].transpose((1,2,0)).astype(np.int32), scale=True) # plot the 1st original image
plot_image(output[0].transpose((1,2,0)).astype(np.int32), scale=True) # plot the output for the 1st image
```

```
(3, 280, 320)
(3, 140, 160)
```





Average pooling

```
output = torch.nn.functional.avg_pool2d(input = batch_tensor,
                                        kernel_size=(2,2), stride= (2,2))
output = output.numpy()
print(batch[0].shape)
print(output[0].shape)
plot_image(batch[0].transpose((1,2,0)).astype(np.int32), scale=True) # plot the 1st original image
plot_image(output[0].transpose((1,2,0)).astype(np.int32), scale=True) # plot the output for the 1st image
```

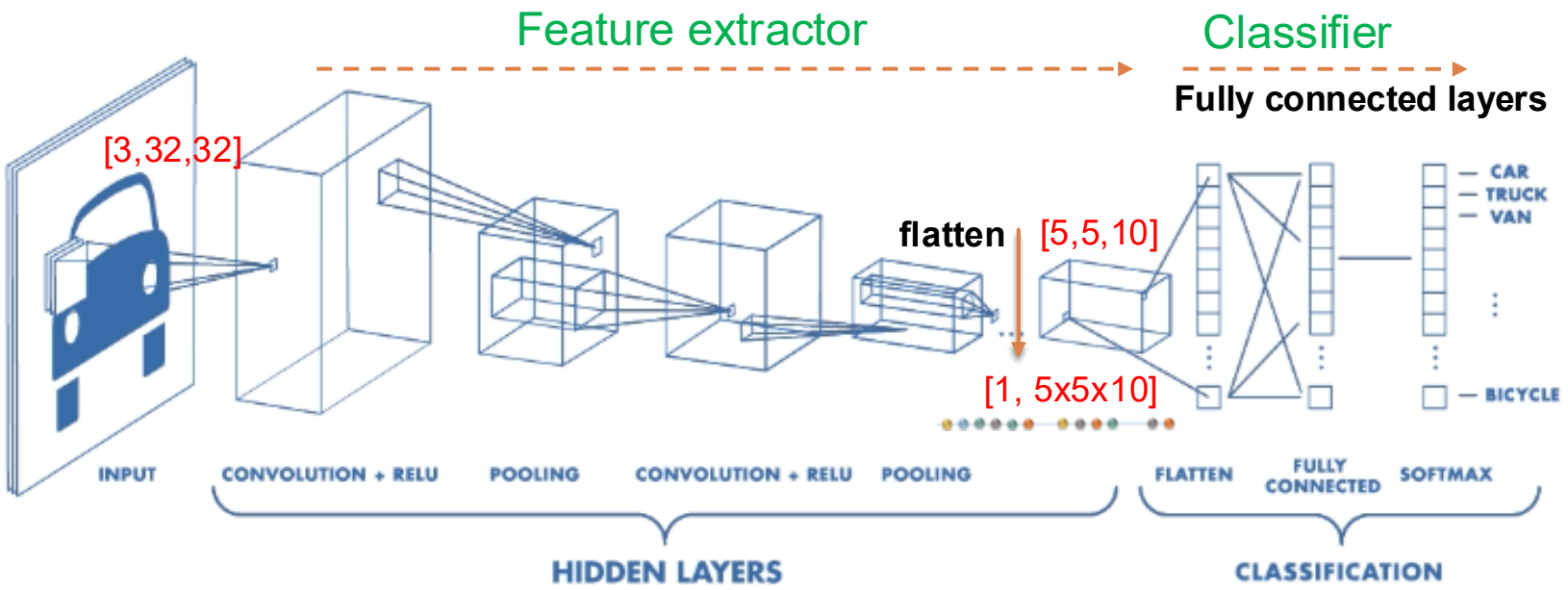
```
(3, 280, 320)
(3, 140, 160)
```





Fully connected layer

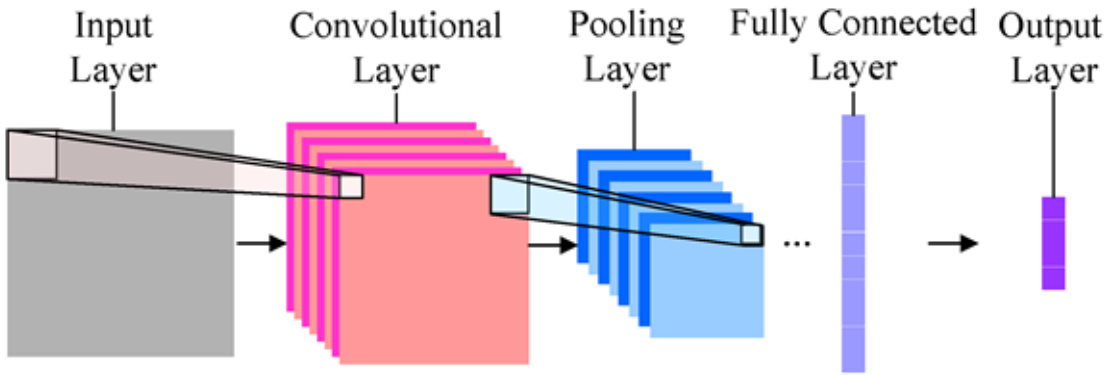
Fully connected layer



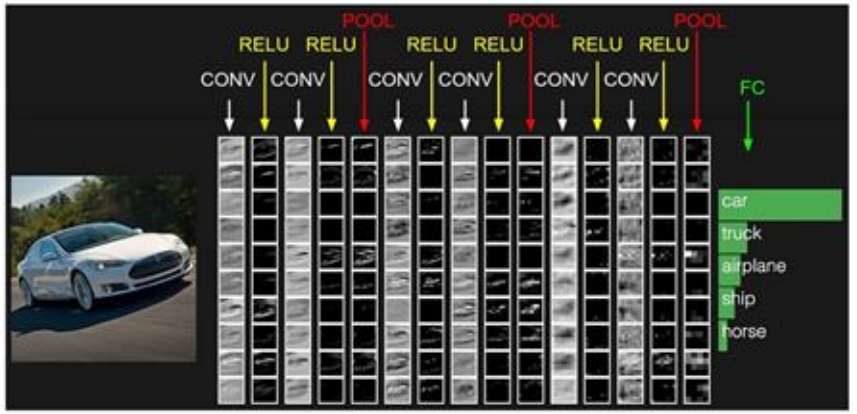
- The last tensor is **flattened,** and **some fully connected layers** are added to classify the input.
  - The last *3D tensor* **[5,5,10]** → 1 **flat layer** with **5x5x10= 250** neurons

Put them all together

General architecture of CNNs

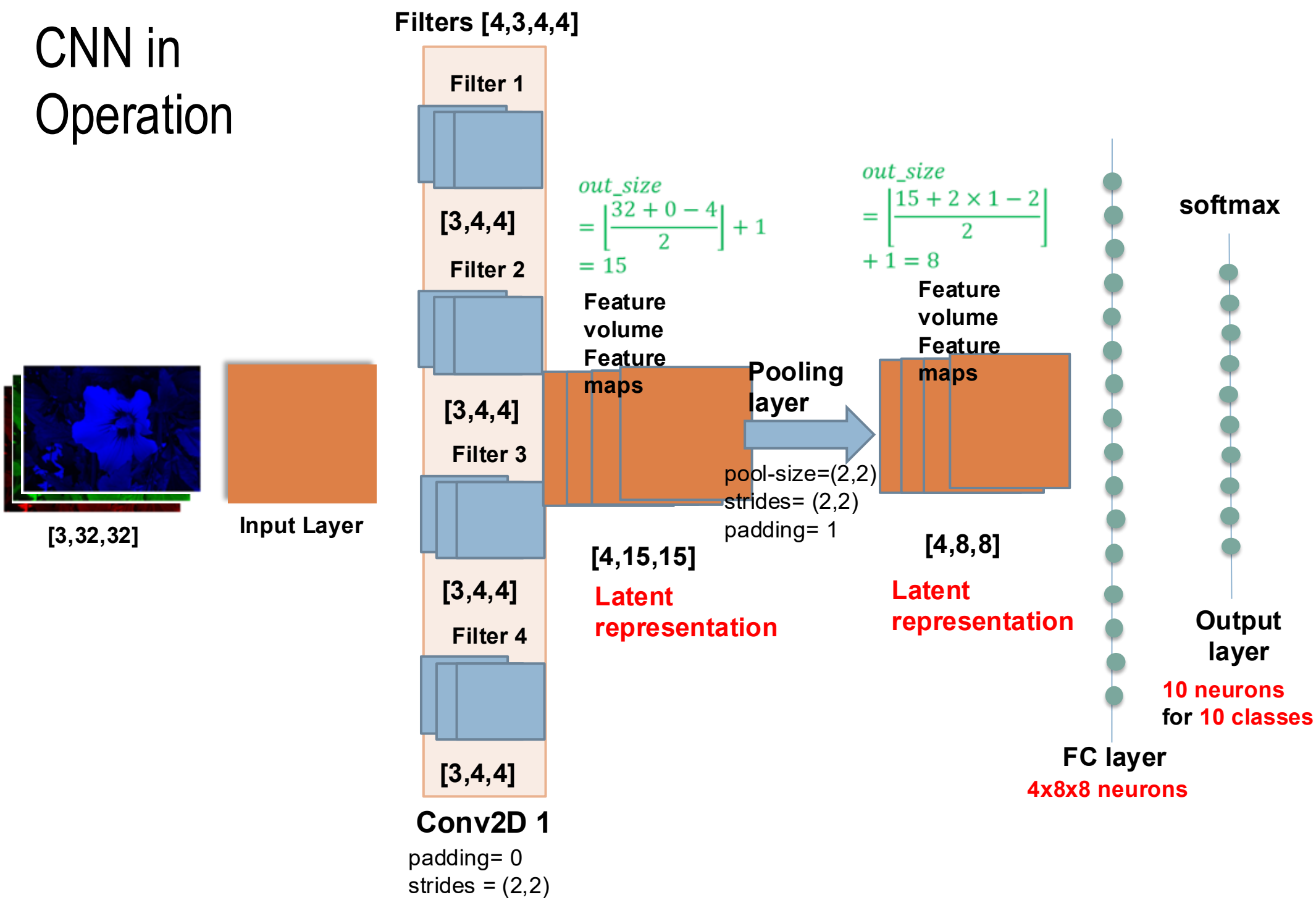


- Convolution stage
- Pooling stage
- Fully connected stage

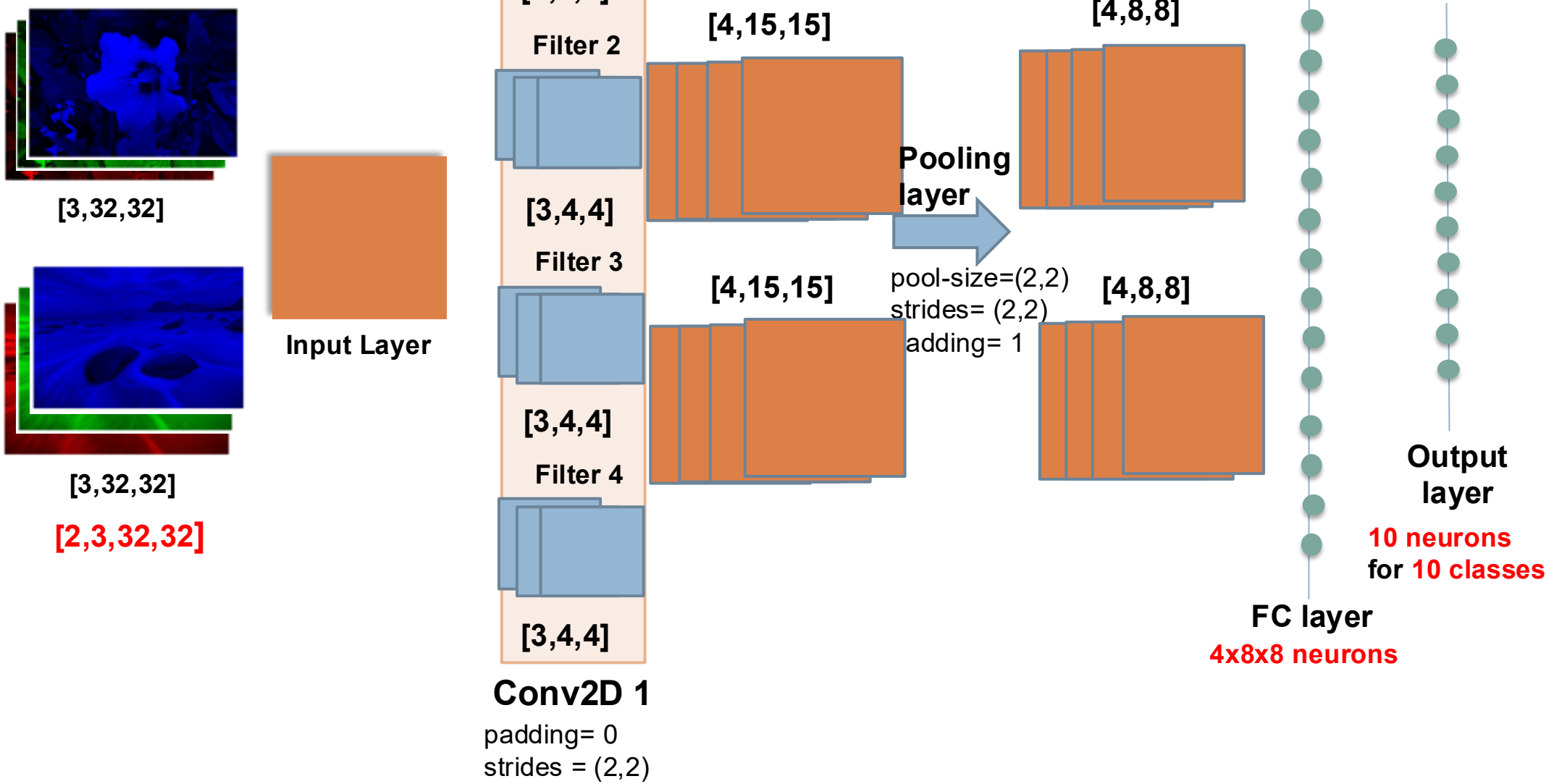


"CS231n: Convolutional Neural Networks for Visual Recognition", Stanford, 2016

CNN in Operation

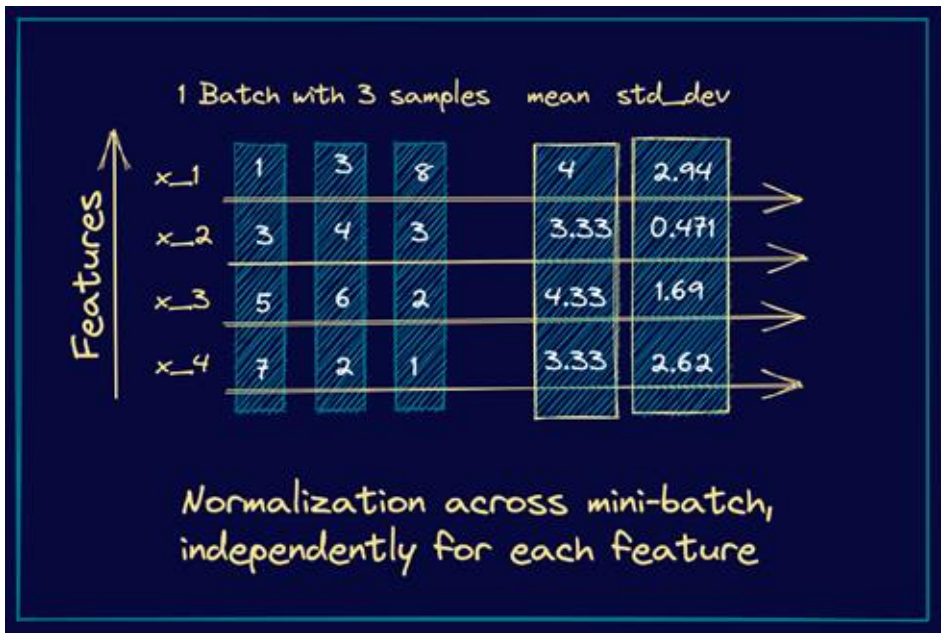


CNN in Operation



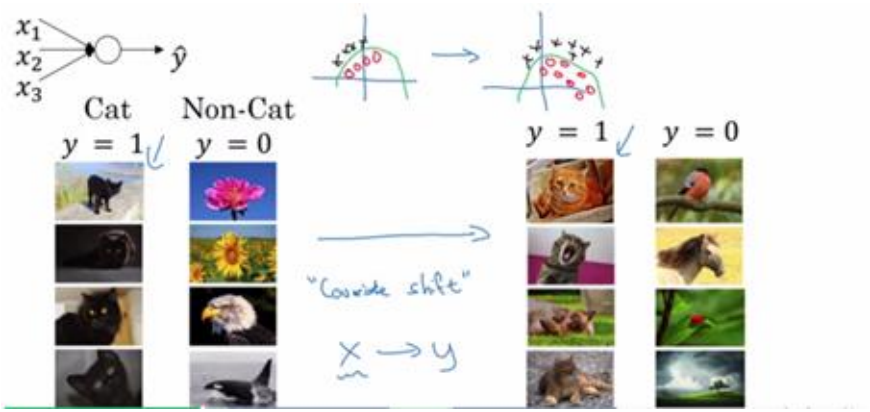

Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
Where?
HOW?

Batch Normalization Layer



Covariate shift vs internal covariate shift

Covariate shift problem

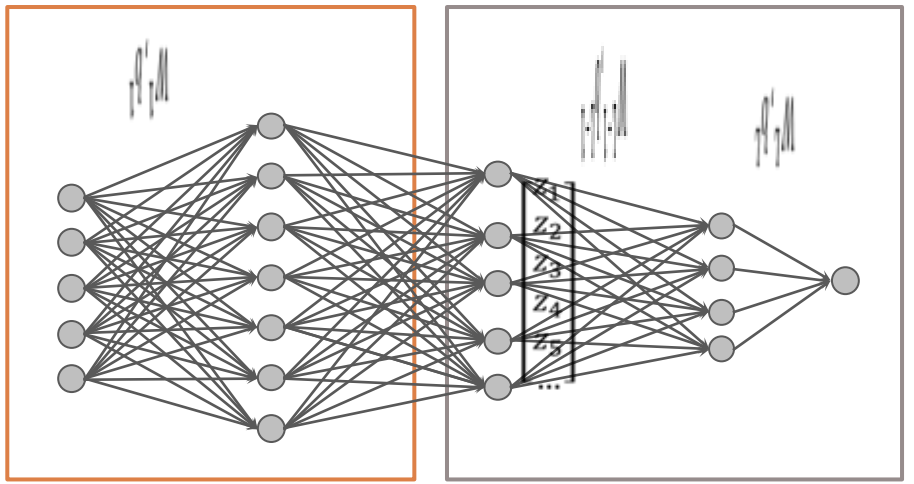


(Source: DL course Andrew Ng)

Covariate shift problem

- The **distribution (nature) of training data** is different from **that of testing data.**

Internal covariate shift

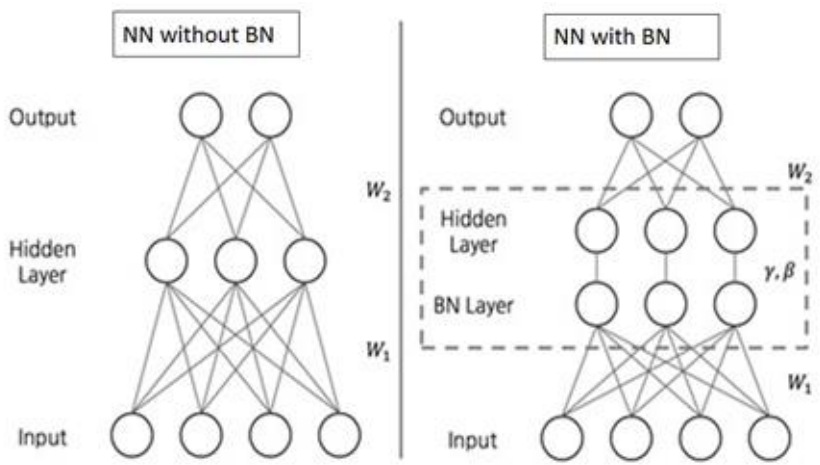


- The **distribution** of $[z_1, z_2, \dots, z_5]$ changes due to
  - The **updates** of $[W^1, b^1], [W^2, b^2]$
  - The **changes** in **mini-batches**

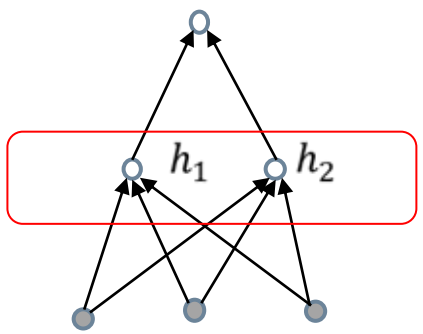
Batch Normalization

1. Cope with internal covariate shift
2. Reduce gradient vanishing/exploding
3. Reduce overfitting
4. Make training more stable
5. Converge faster
    1. Allow us to train with bigger learning rate

- Let $z = W^k h^k + b^k$ be the mini-batch before activation. We compute the normalized $\hat{z}$ as
    - $\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ where ϵ is a small value such as $1e^{-7}$
    - $\mu_B = \frac{1}{b}\sum_{i=1}^{b} z_i$ is the empirical mean
    - $\sigma_B^2 = \frac{1}{b}\sum_{i=1}^{b}(z_i - \mu_B)^2$ is the empirical variance
- We scale the normalized $\hat{z}$
    - $z_{BN} = \gamma\hat{z} + \beta$ where $\gamma, \beta > 0$ are two learnable parameters (i.e., scale and shift parameters)
- We then apply the activation to obtain the next layer value
    - $h^{k+1} = \sigma(z_{BN})$



(Source: medium.com)



0.1	0.2
…	…
0.5	0.4

Mini-batch size

Standardize column-wise

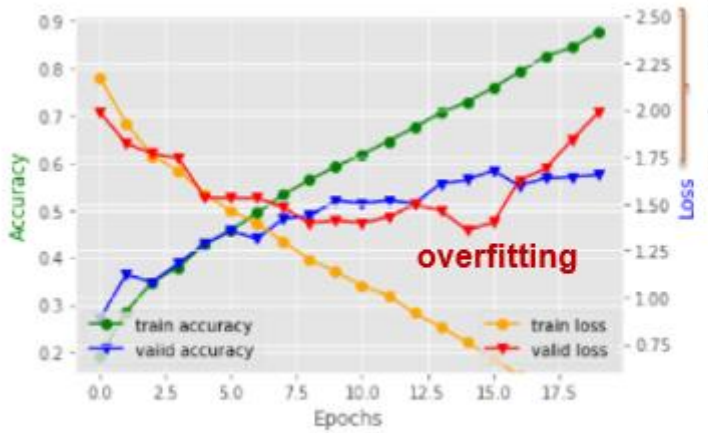
Batch Normalization

Testing Phase

- At testing time, let's say we only want to test on **a single input** x

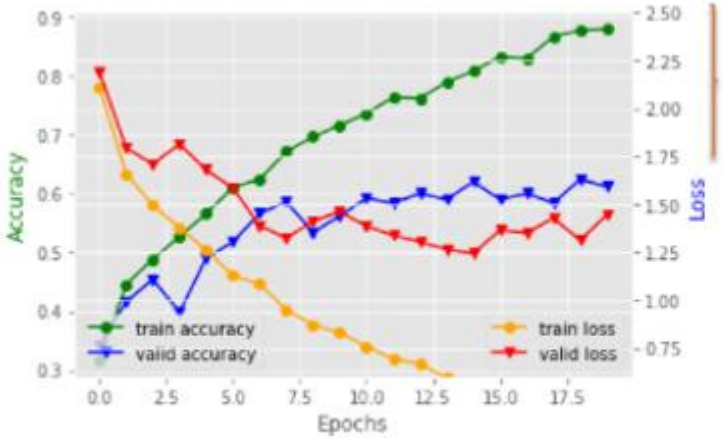
- Hence, we **don't have** a set of mini-batch samples to compute mean and standard deviation.

- So, we replace the **mini-batch** μ_B and σ_B with **running averages** of $\tilde{\mu}_B$ and $\tilde{\sigma}_B$ computed during the training process.
  - $\tilde{\mu}_B = \theta\tilde{\mu}_B + (1-\theta)\mu_B$
  - $\tilde{\sigma}_B = \theta\tilde{\sigma}_B + (1-\theta)\sigma_B$
  - $0 < \theta < 1$ is the momentum decay



87%
Bigger gap
57%

Without batch norm



88%
Smaller gap
62%

With batch norm

Implementation of BatchNorm2D in PyTorch

Self-study

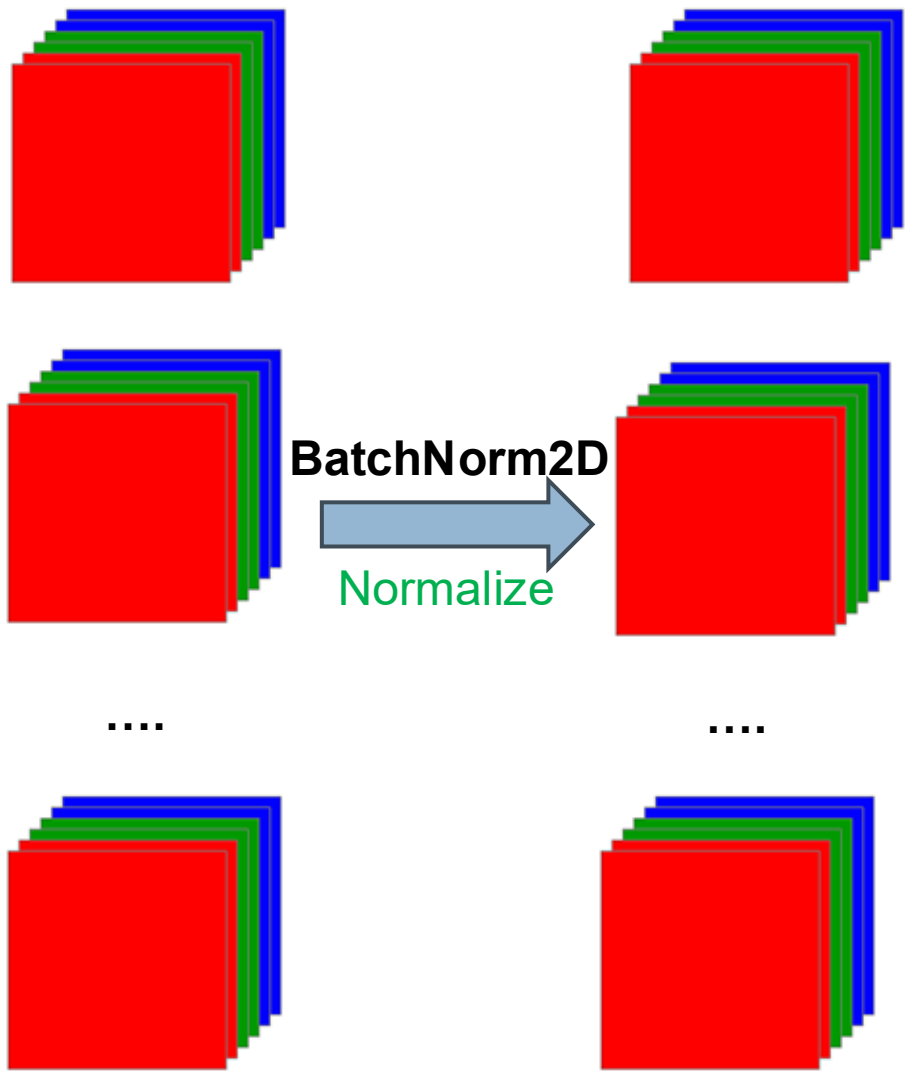
```python
def batch_norm(input, running_mean, running_var, weight=None, bias=None,
               training=False, momentum=0.1, eps=1e-5):
    if training:
        # Compute the mean and variance of the input
        mean = input.mean([0, 2, 3], keepdim=True)
        var = input.var([0, 2, 3], unbiased=False, keepdim=True)

        # Update the running mean and variance
        with torch.no_grad():
            running_mean.mul_(momentum)
            running_mean.add_((1 - momentum) * mean.squeeze())
            running_var.mul_(momentum)
            running_var.add_((1 - momentum) * var.squeeze())
    else:
        mean = running_mean
        var = running_var

    # Normalize the input
    input = (input - mean) / (torch.sqrt(var + eps))

    if weight is not None and bias is not None:
        input = input * weight + bias

    return input
```

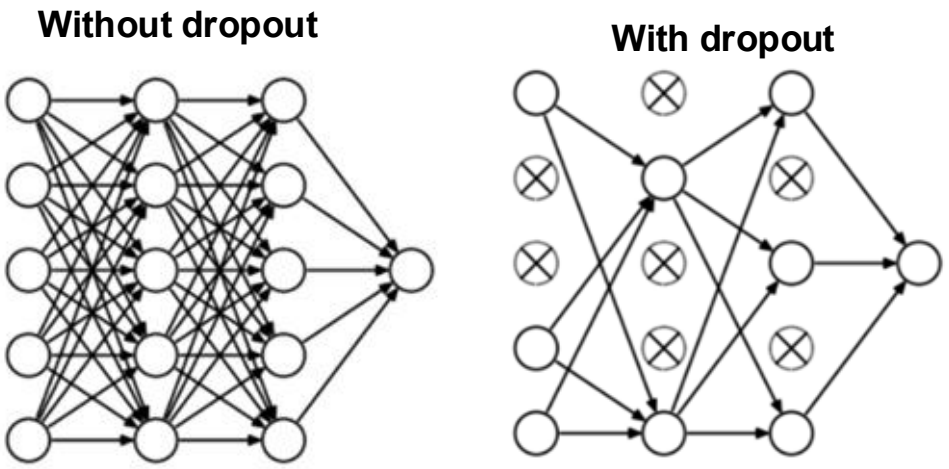


Dropout Layer

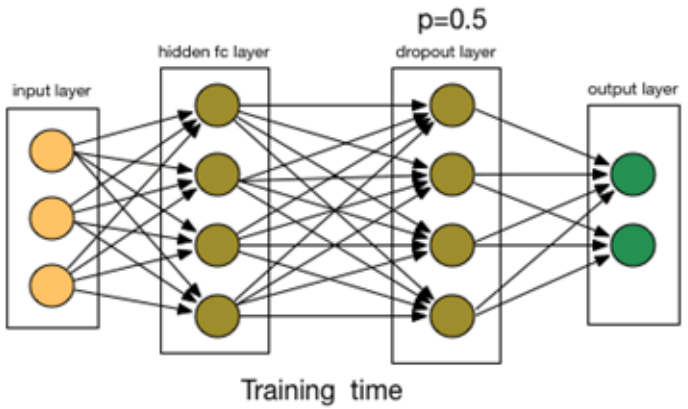


Dropout

Reduce Overfitting



(Source: Analytics Vidhya)



(Source: chatbotslife)

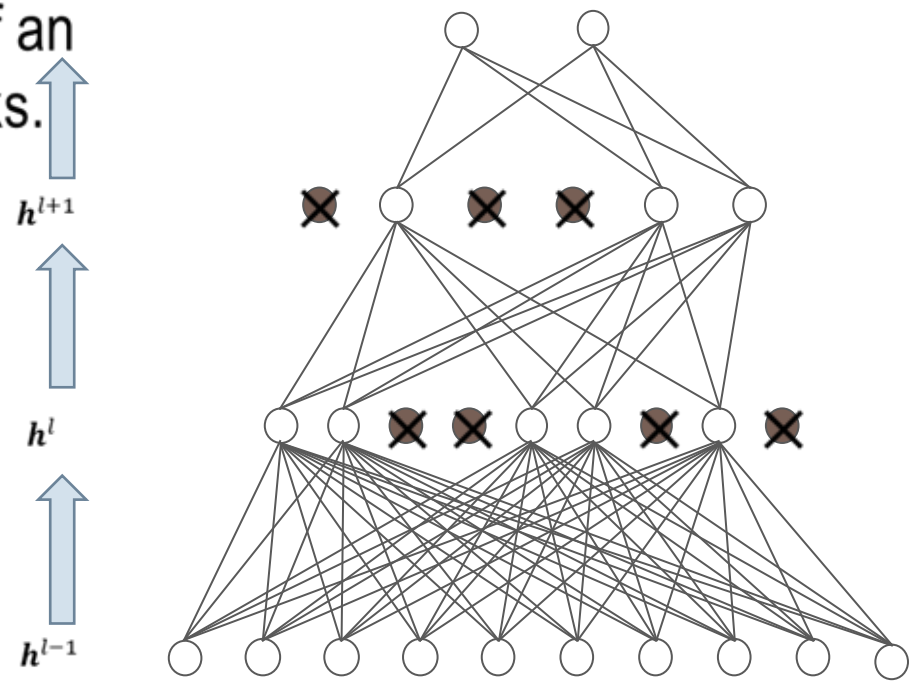
- This is a **cheap technique** to reduce model capacity
  - Reduce overfitting
- In each iteration, at each layer, **randomly choose** some neurons and **drop all connections** from these neurons
  - dropout_rate= 1 – keep_prob

Dropout

Reduce Overfitting

- Computationally efficient
- Can be considered a **bagged ensemble** of an exponential number (2^N) of neural networks.
- **Training**

$$\boldsymbol{r} \sim \text{Bernoulli}(\mu)$$
$$\tilde{\boldsymbol{h}}^l = \boldsymbol{h}^l \odot \boldsymbol{r}$$
$$\boldsymbol{h}^{l+1} = \sigma\left(W^{(l)^\top} \tilde{\boldsymbol{h}}^l + \boldsymbol{b}^l\right)$$

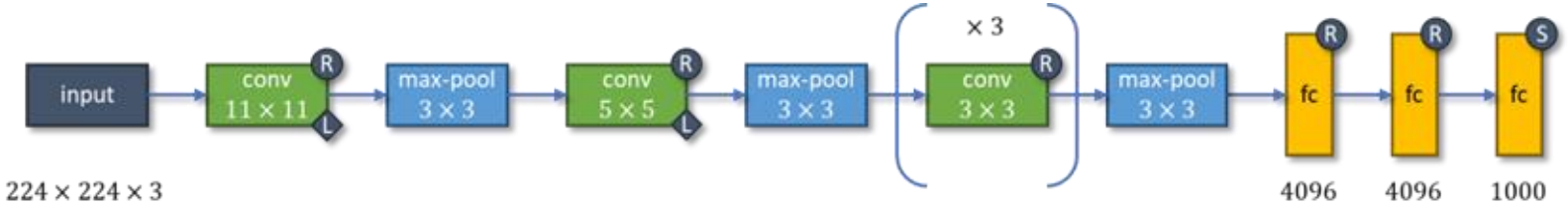


- **Testing**
  - **No dropout** (dropout_rate =0).

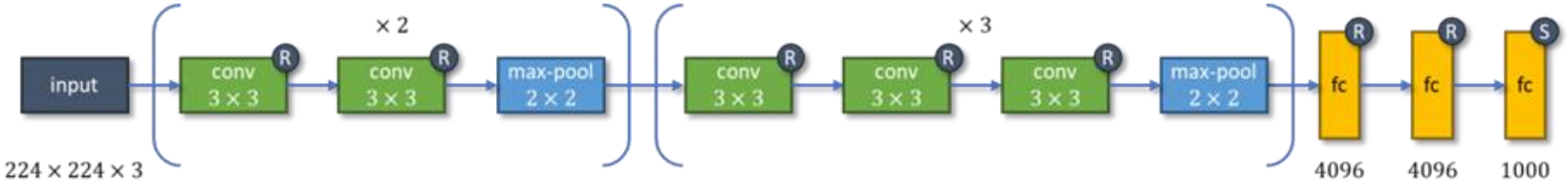


Some popular CNN architectures

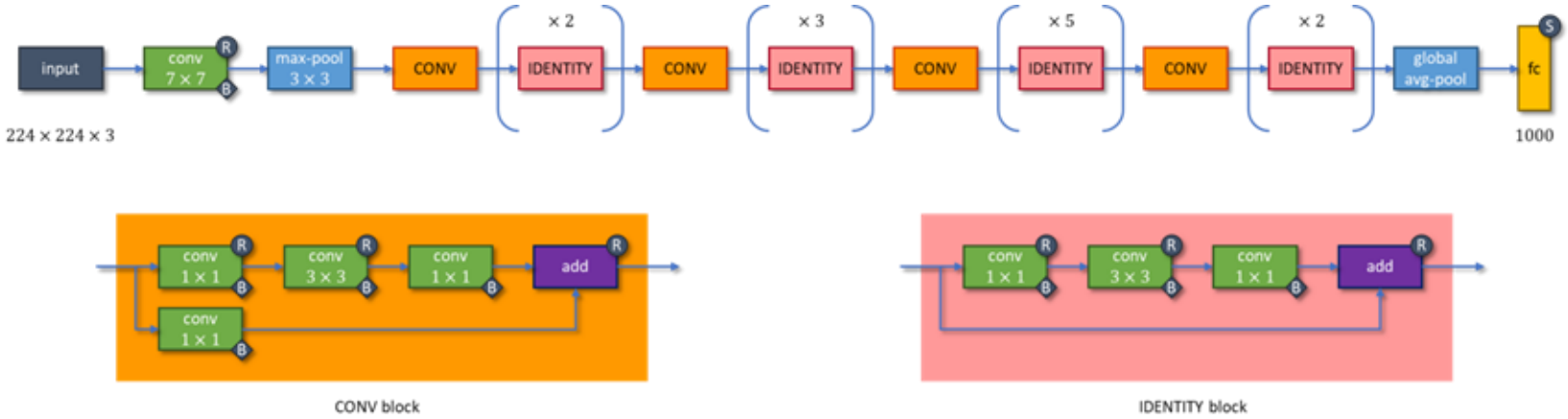
Network Architectures



AlexNet



VGG16

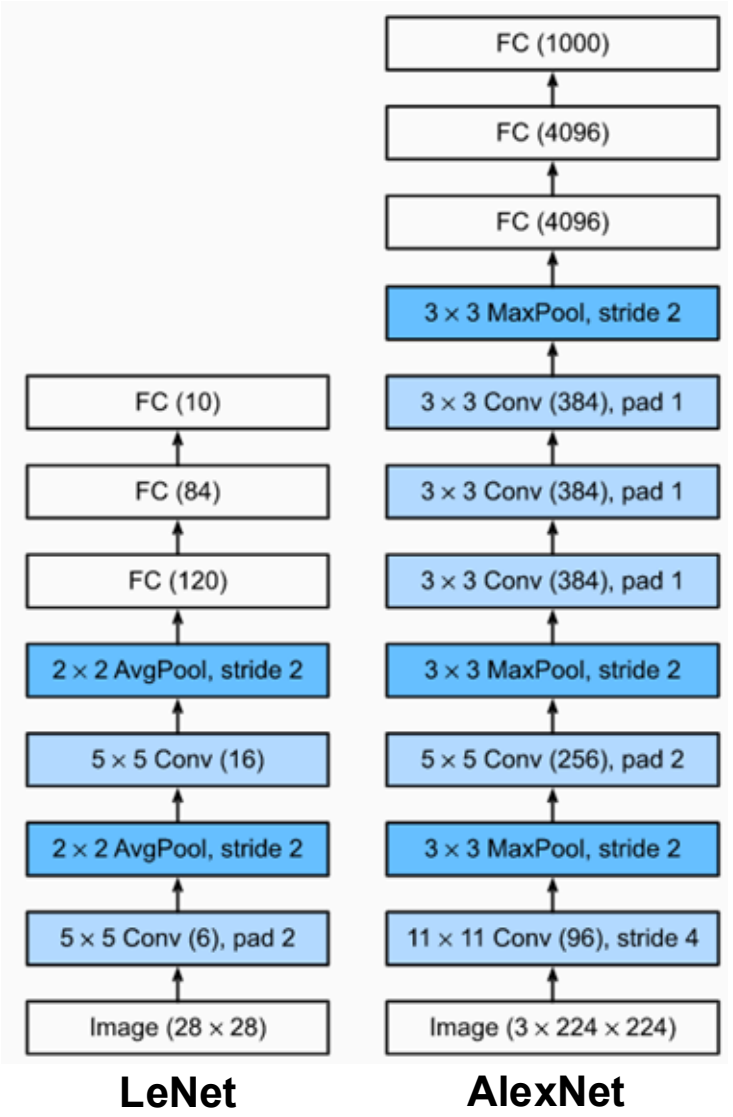


ResNet

LeNet and AlexNet

- The **design philosophies** of AlexNet and LeNet are **very similar,** but there are also **significant differences.**
  - AlexNet is much **deeper than** the comparatively small LeNet5.
  - AlexNet consists of **eight layers**
    - **Five convolutional** layers, **two fully-connected** hidden layers, and **one fully-connected output** layer.
  - AlexNet used the **ReLU** instead of the **sigmoid**
  - In AlexNet's first layer, the convolution filter shape is **11×11** because **objects in ImageNet** data tend to **occupy more pixels.**

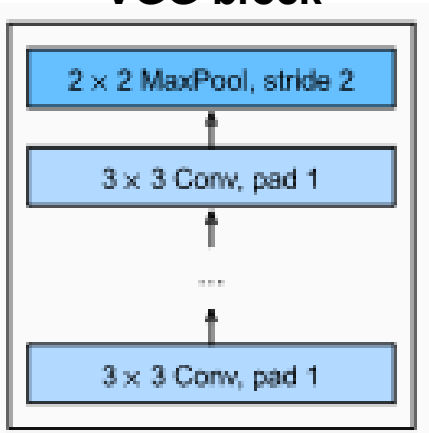
(Source:Dive into DL)



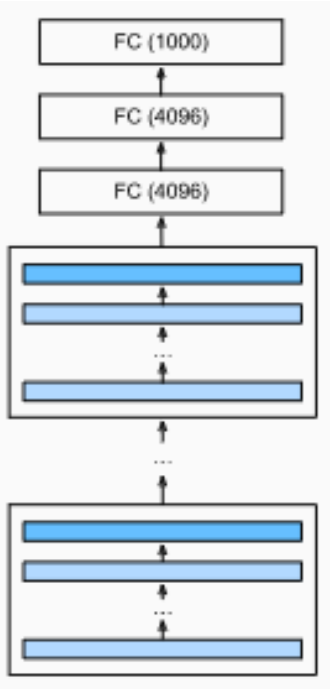
VGG Network

- **Won 2nd in ImageNet** competition in **2014**
- Moving from thinking in terms of **individual neurons** ☐ **whole layers** ☐ **blocks** (repeating patterns of layers)
- **VGG block**
  - One VGG block consists of a **sequence of convolutional layers, followed by a max pooling** layer for spatial downsampling.
- **VGG network**
  - Consists of **many VGG blocks,** followed by **fully-connected layers**
  - The original VGG network had **5 convolutional blocks**
    - The **first two** have **one convolutional layer** each and the **latter three** contain **two convolutional layers** each.
    - The **first block** has **64 output channels** and **each subsequent block** doubles **the number of output channels,** until that number **reaches 512.**

VGG block



VGG network



(Source: Dive into DL)

MiniVGG for Cifar10

Our Tutorial

Layer Type	Output Size	Filter Size / Stride
INPUT IMAGE	$32 \times 32 \times 3$	
CONV	$32 \times 32 \times 32$	$3 \times 3, K = 32$
ACT	$32 \times 32 \times 32$	
BN	$32 \times 32 \times 32$	
CONV	$32 \times 32 \times 32$	$3 \times 3, K = 32$
ACT	$32 \times 32 \times 32$	
BN	$32 \times 32 \times 32$	
POOL	$16 \times 16 \times 32$	2×2
DROPOUT	$16 \times 16 \times 32$	
CONV	$16 \times 16 \times 64$	$3 \times 3, K = 64$
ACT	$16 \times 16 \times 64$	
BN	$16 \times 16 \times 64$	
CONV	$16 \times 16 \times 64$	$3 \times 3, K = 64$
ACT	$16 \times 16 \times 64$	
BN	$16 \times 16 \times 64$	
POOL	$8 \times 8 \times 64$	2×2
DROPOUT	$8 \times 8 \times 64$	
FC	512	
ACT	512	
BN	512	
DROPOUT	512	
FC	10	
SOFTMAX	10	

```python
def create_vgg():
    vgg_model = nn.Sequential(
        #nn.LazyConv2d(32, kernel_size=3, padding=1),
        nn.Conv2d(3, 32, kernel_size=3, padding= 1), #[32,32,32]
        nn.BatchNorm2d(32),
        nn.ReLU(),
        #nn.LazyConv2d(32, kernel_size=3, padding=1),
        nn.Conv2d(32, 32, kernel_size=3, padding=1), #[32,32,32]
        nn.BatchNorm2d(32),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2), #down-sample by two #[32,16,16]
        nn.Dropout(p=0.25),

        #nn.LazyConv2d(64, kernel_size=3, padding=1),
        nn.Conv2d(32, 64 , kernel_size=3, padding=1), #[64,16,16]
        nn.BatchNorm2d(64, momentum=0.1),
        nn.ReLU(),
        #nn.LazyConv2d(64, kernel_size=3, padding=1)
        nn.Conv2d(64, 64, kernel_size=3, padding=1), #[64,16,16]
        nn.BatchNorm2d(64, momentum=0.1),
        nn.ReLU(),
        nn.LazyConv2d(64, kernel_size=3, padding=1),
        nn.BatchNorm2d(64, momentum=0.1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2), #down-sample by two [64,8,8]
        nn.Dropout(p=0.25),

        nn.Flatten(1), #64x8x8
        #nn.Linear(64x8x8, 512)
        nn.LazyLinear(512),
        nn.ReLU(),
        #nn.LazyLinear(10)
        nn.Linear(512, 10),
      )
    return vgg_model
```



60,000 images
10 classes

Thanks for your attention!

